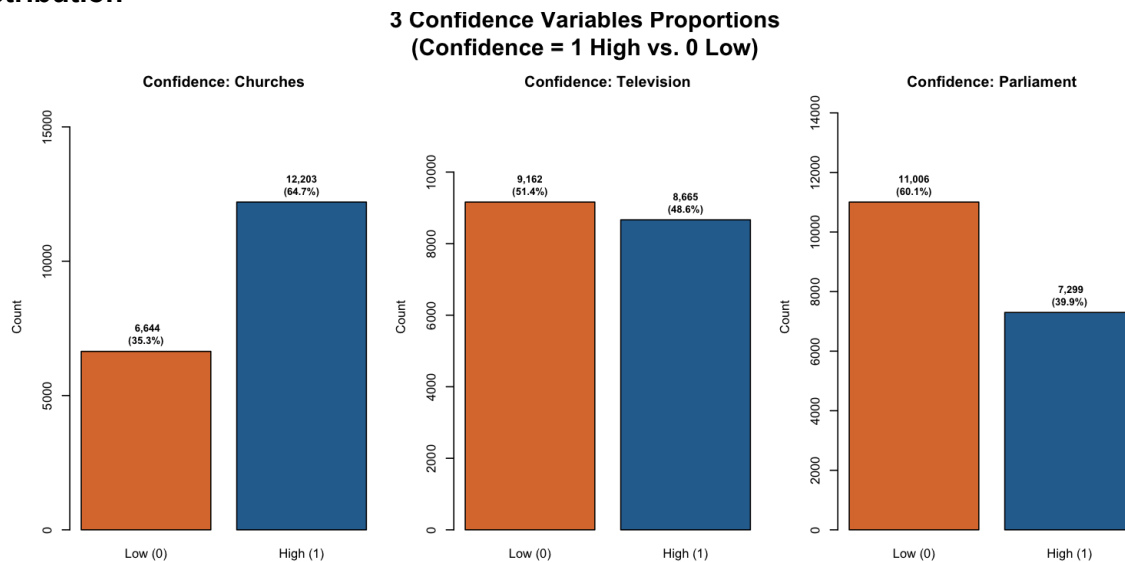


No content generated by AI technologies has been used in this assessment.

Q1: Data Exploration

The dataset contains 35 columns and 20,000 rows, covering 58 countries across 7 survey waves.

Target distribution

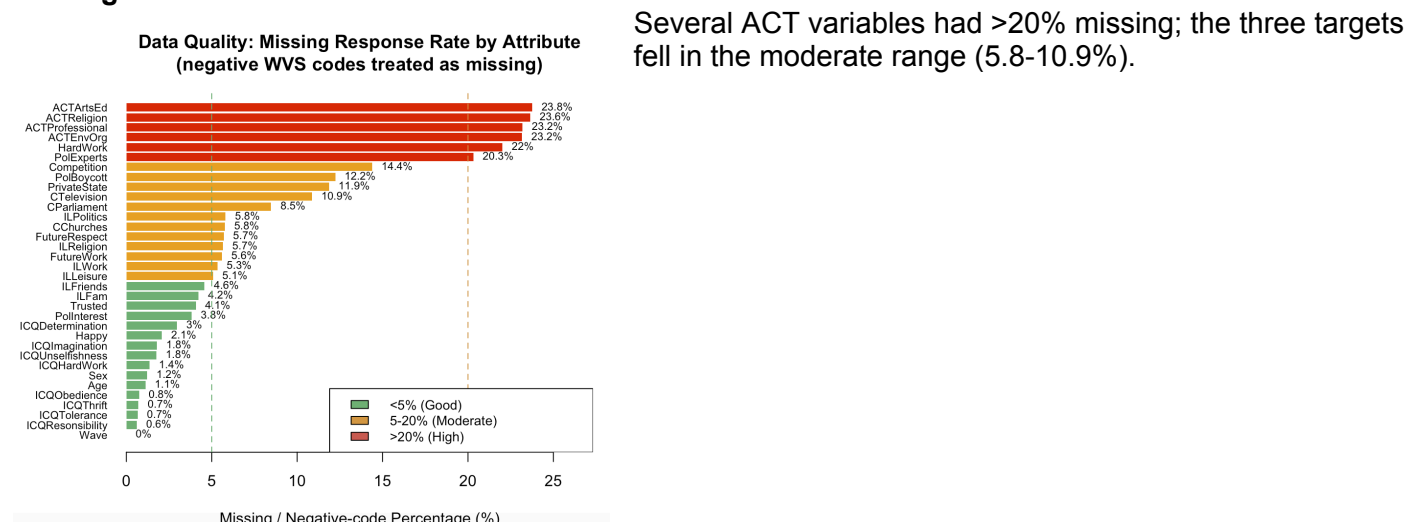


- CChurches (low 35.4%; high 64.7%): strongly skewed toward high trust.
- CTelevision (low 51.4%; high 48.6%): near 50/50 balance.
- CParliament (low 60.1%; high 39.9%): skewed toward low trust.

Predictor descriptions

30 predictors cover values (IL), child quality (ICQ), group memberships (ACT), well-being, political attitudes (Pol), and demographics; scales range binary to 1-10. Age is right-skewed (median 40); Sex is near-balanced (48.7/51.3%).

Missing values



Several ACT variables had >20% missing; the three targets fell in the moderate range (5.8-10.9%).

All predictors were retained with complete-case filtering to preserve potential signal. Country is excluded from classification.

Q2: Data preprocessing

Three steps were applied:

1. **WVS negative codes were converted to NA.** Negative codes were replaced with NA across all numeric columns.
2. **Complete-case filtering** was applied to remove rows containing any NA.
3. **Categorical variables converted to factors:** 3 targets, Country, Wave, Sex

Sample size dropped from 20,000 to 9,756 (~51% loss).

Q3: 70/30 Train/TestSplit

The cleaned dataset (n = 9,756) are split into WD.train: 6829 x 35; WD.test: 2927 x 35 after 70/30 split.

Q4: Model Implementation

For each target, five classifiers were trained at default settings. The remaining confidence variables and Country were excluded (Country exceeded tree::tree()'s 32-level cap).

- Decision Tree (DT): tree::tree()
- Naïve Bayes (NB): e1071::naiveBayes()
- Bagging: adabag::bagging()
- Boosting: adabag::boosting()
- Random Forest (RF): randomForest::randomForest()

Example: CChurches models

```
tree_churches <- tree(CChurches ~ . - CTelevision - CParliament - Country, data = WD.train)
nb_churches <- naiveBayes(CChurches ~ . - CTelevision - CParliament - Country, data = WD.train)
bag_churches <- bagging(CChurches ~ . - CTelevision - CParliament - Country, data = WD.train)
boost_churches <- boosting(CChurches ~ . - CTelevision - CParliament - Country, data = WD.train)
rf_churches <- randomForest(CChurches ~ . - CTelevision - CParliament - Country, data = WD.train)
```

Q5 Confusion Matrix & Performance Matrix

CChurches

	Decision Tree		Naive Bayes		Bagging		Boosting		Random Forest	
	Pred 0	Pred 1	Pred 0	Pred 1	Pred 0	Pred 1	Pred 0	Pred 1	Pred 0	Pred 1
True 0	681	419	698	402	681	419	670	430	655	445
True 1	298	1,529	376	1,451	298	1,529	276	1,551	262	1,565

All five classifiers achieved similar F1 (0.789-0.816). RF achieved the strongest overall performance. Decision Tree and Bagging produced identical confusion matrices and F1 (0.810), both dominated by ILReligion.

Model	Accuracy	Precision	Recall	F1
Tree	0.755	0.785	0.837	0.810
NaiveBayes	0.734	0.783	0.794	0.789
Bagging	0.755	0.785	0.837	0.810
Boosting	0.759	0.783	0.849	0.815
RandomForest	0.758	0.779	0.857	0.816

CTelevision

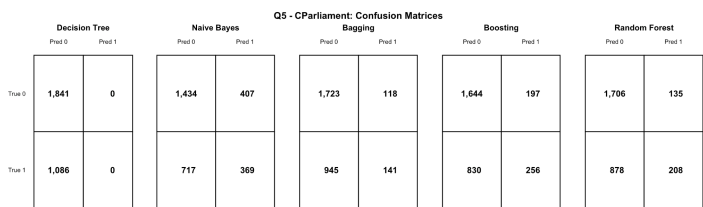
	Decision Tree		Naive Bayes		Bagging		Boosting		Random Forest	
	Pred 0	Pred 1	Pred 0	Pred 1	Pred 0	Pred 1	Pred 0	Pred 1	Pred 0	Pred 1
True 0	1,591	0	951	640	1,414	177	1,142	449	1,198	393
True 1	1,336	0	610	726	1,068	268	739	597	801	535

Performance fell between the other two targets

Ensembles showed high precision but low recall.

Model	Accuracy	Precision	Recall	F1
Tree	0.544	NaN	0.000	NaN
NaiveBayes	0.573	0.531	0.543	0.537
Bagging	0.575	0.602	0.201	0.301
Boosting	0.594	0.571	0.447	0.501
RandomForest	0.592	0.577	0.400	0.473

CParliament



Performance was the **weakest** of the three targets

Ensemble methods exhibited the same many TPs missed pattern in CTelevision.

Model	Accuracy	Precision	Recall	F1
Tree	0.629	NaN	0.000	NaN
NaiveBayes	0.616	0.476	0.340	0.396
Bagging	0.637	0.544	0.130	0.210
Boosting	0.649	0.565	0.236	0.333
RandomForest	0.654	0.606	0.192	0.291

Interesting observation

Decision Tree classified every instance as Low for both CParliament (one PolInterest split, but both leaves Low-majority) and CTelevision (no splits - no predictor sufficiently reduced impurity), yielding undefined F1.

Although CParliament had the lowest F1, it showed the highest ensemble accuracy, largely driven by correct predictions of the majority Low class despite weak detection of true High cases.

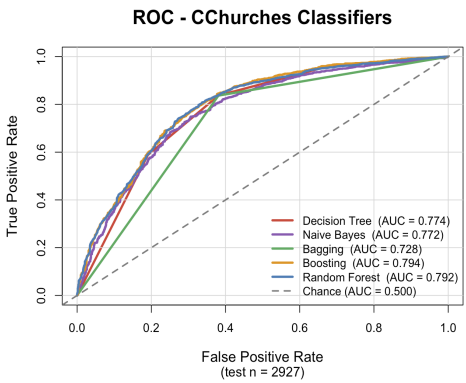
Q6: ROC and AUC Comparison Across Classifiers

Across the three targets, CChurches shows the strongest predictability, while CTelevision and CParliament cluster at substantially lower AUC ranges. Most classifiers outperform random guessing across all three variables, with one notable exception: the Decision Tree on CTelevision yields AUC = 0.500.

- CChurches: Boosting wins (0.794).
- CTelevision: RF wins (0.622); DT at chance level. DT collapses because no predictor provides sufficient information gain to justify any split.
- CParliament: RF wins (0.648); DT (0.558) found at least one informative split (PolInterest).

ROC shape comparison: For CChurches, all ROC curves except Bagging rise steeply at low FPR, indicating strong true-positive detection, while Bagging shows a flatter curve. In contrast, the curves for CTelevision and CParliament lie close to the diagonal with considerable overlap, suggesting minimal performance differences between classifiers.

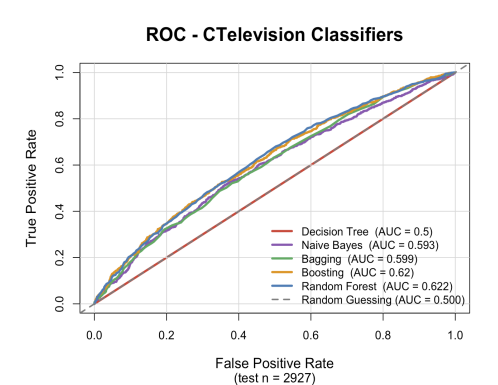
CChurches



CChurches Classifiers: AUC Comparison

Classifier	AUC
Decision Tree	0.774
Naive Bayes	0.772
Bagging	0.728
Boosting	0.794
Random Forest	0.792

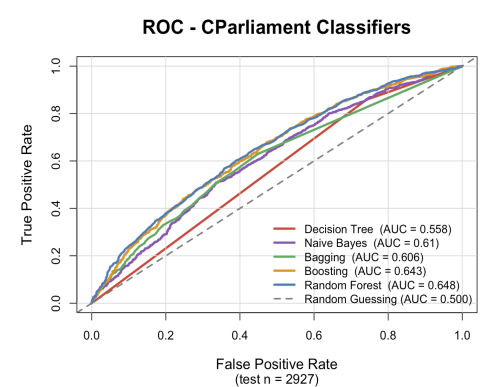
CTelevision



CTelevision Classifiers: AUC Comparison

Classifier	AUC
Decision Tree	0.500
Naive Bayes	0.593
Bagging	0.599
Boosting	0.620
Random Forest	0.622

CParliament

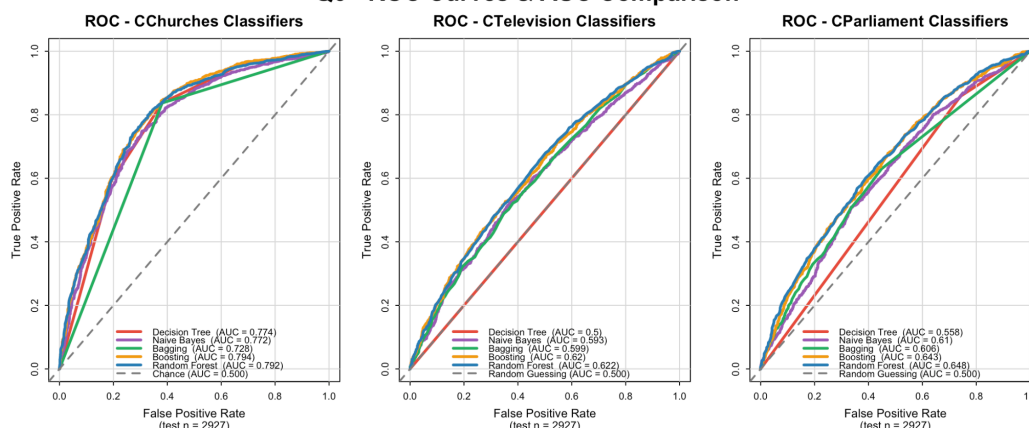


CParliament Classifiers: AUC Comparison

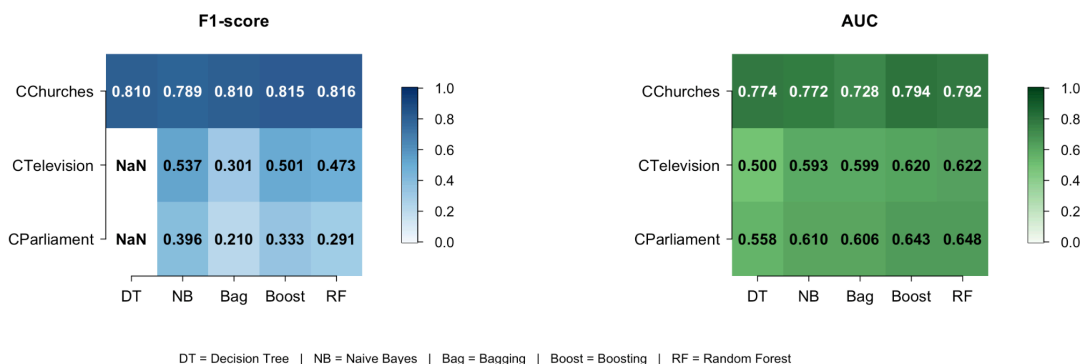
Classifier	AUC
Decision Tree	0.558
Naive Bayes	0.610
Bagging	0.606
Boosting	0.643
Random Forest	0.648

Q7: Comparison of classification performance

Q6 - ROC Curves & AUC Comparison



Q7: Cross-Classifier Comparison (F1 & AUC)



Single best classifier. No classifier consistently outperformed all others across the three targets. On CChurches, all five classifiers F1 result are similar. On CTelevision and CParliament, NB attains the highest F1, but RF and Boosting dominate on AUC across all three targets, indicating that ensembles produce the most reliable probability rankings.

Worst ensemble classifier. Bagging was the weakest ensemble on every target.

Best varies because each model's inductive bias interacts differently with each target's feature strength and class balance. Worst is stable - probably due to bagging's bootstrap sampling mechanism unlike feature resampling (RF) and iterative correction (Boosting).

Q8: Consistently Important Variables Across Ensemble Models

Rankings (not raw scores) were averaged across the three ensembles to handle scale incompatibility, with the limitation of losing magnitude information.

CChurches

Q8: Variable Importance: CChurches

Variables ranked by average ranking across ensemble models; top 5 highlighted in yellow

Variable	Bagging	Boosting	RandomForest	BagRank	BoostRank	RFRank	AvgRank
ILReligion	100	82.89	640.28	1.0	1	1	1.00
Age	0	1.33	261.21	16.5	3	2	7.17
ACTReligion	0	4.63	156.66	16.5	2	5	7.83
PrivateState	0	0.75	166.87	16.5	6	3	8.50
HardWork	0	0.67	166.16	16.5	9	4	9.83
ILPolitics	0	0.93	104.65	16.5	4	9	9.83
Wave	0	0.69	133.85	16.5	8	7	10.50
ILLeisure	0	0.60	90.67	16.5	10	11	12.50
PolExperts	0	0.51	107.84	16.5	13	8	12.50
PolInterest	0	0.53	102.56	16.5	12	10	12.83
FutureRespect	0	0.72	78.23	16.5	7	16	13.17
Competition	0	0.26	153.77	16.5	20	6	14.17
ILFriends	0	0.44	83.86	16.5	16	13	15.17
ILWork	0	0.46	80.97	16.5	15	14	15.17
Happy	0	0.33	86.94	16.5	18	12	15.50
ICQThrift	0	0.54	50.59	16.5	11	19	15.50
FutureWork	0	0.32	78.70	16.5	19	15	16.83
ILFam	0	0.86	37.38	16.5	5	30	17.17
PolBoycott	0	0.25	69.09	16.5	21	17	18.17
ICQTolerance	0	0.41	49.63	16.5	17	22	18.50
ICQHardWork	0	0.22	53.31	16.5	23	18	19.17
Trusted	0	0.47	44.92	16.5	14	27	19.17
ICQDetermination	0	0.15	50.55	16.5	25	20	20.50
ICQResponsibility	0	0.23	44.92	16.5	22	26	21.50
ICQObedience	0	0.11	49.92	16.5	28	21	21.83
ACTArtsEd	0	0.08	48.98	16.5	29	24	23.17
ICQUnselfishness	0	0.08	49.28	16.5	30	23	23.17
ACTEnvOrg	0	0.19	35.80	16.5	24	31	23.83
ACTProfessional	0	0.13	43.74	16.5	27	28	23.83
ICQImagination	0	0.15	42.69	16.5	26	29	23.83
Sex	0	0.04	47.14	16.5	31	25	24.17

ILReligion was consistently the most important predictor across all ensemble models.

CParliament

Q8: Variable Importance: CParliament

Variables ranked by average ranking across ensemble models; top 5 highlighted in yellow

Variable	Bagging	Boosting	RandomForest	BagRank	BoostRank	RFRank	AvgRank
PolInterest	46.63	33.76	138.91	1	1	6	2.67
HardWork	2.47	6.01	195.80	6	3	3	4.00
ILPolitics	20.57	7.74	127.04	2	2	8	4.00
PrivateState	1.88	5.04	205.59	7	4	2	4.33
Age	0.91	4.97	328.71	9	5	1	5.00
Wave	0.68	2.64	157.50	10	13	5	9.33
FutureRespect	3.98	3.25	86.77	4	10	16	10.00
Happy	0.18	4.77	103.19	16	6	11	11.00
ILReligion	0.10	3.73	118.21	19	8	9	12.00
PolBoycott	0.97	2.73	83.33	8	12	17	12.33
ILLeisure	0.34	1.31	114.12	13	16	10	13.00
Trusted	16.10	4.54	52.09	3	7	29	13.00
ILFriends	0.11	2.77	99.13	17	11	13	13.67
ACTEnvOrg	3.35	3.31	51.01	5	9	30	14.67
Competition	0.00	2.09	187.73	28	14	4	15.33
ILWork	0.40	0.67	93.14	12	22	14	16.00
PolExperts	0.00	1.16	132.12	28	18	7	17.67
ACTReligion	0.08	1.20	88.18	22	17	15	18.00
Sex	0.42	1.12	56.16	11	19	26	18.67
ACTArtsEd	0.22	0.58	61.78	14	25	19	19.33
FutureWork	0.09	0.30	99.36	20	30	12	20.67
ICQDetermination	0.20	0.18	64.92	15	31	18	21.33
ICQTolerance	0.06	1.46	57.02	24	15	25	21.33
ACTProfessional	0.10	1.07	55.07	18	20	27	21.67
ICQResponsibility	0.09	0.42	59.91	21	28	22	23.67
ICQHardWork	0.00	0.59	60.24	28	23	21	24.00
ICQThrift	0.00	0.42	61.68	28	27	20	25.00
ICQUnselfishness	0.00	0.58	57.82	28	24	24	25.33
ICQObedience	0.00	0.54	59.63	28	26	23	25.67
ICQImagination	0.07	0.35	54.02	23	29	28	26.67
ILFam	0.00	0.70	34.65	28	21	31	26.67

Political interest (PolInterest) was the strongest overall predictor across the ensemble models.

CTelevision

Q8: Variable Importance: CTelevision

Variables ranked by average ranking across ensemble models; top 5 highlighted in yellow

Variable	Bagging	Boosting	RandomForest	BagRank	BoostRank	RFRank	AvgRank
Wave	6.41	7.97	162.89	4	2	5	3.67
HardWork	6.40	4.68	210.49	5	4	3	4.00
Age	2.59	13.30	357.88	12	1	1	4.67
PolInterest	10.70	4.28	131.23	3	5	7	5.00
PrivateState	2.71	5.62	217.92	11	3	2	5.33
ILPolitics	10.84	4.15	127.33	2	8	8	6.00
ILReligion	25.62	4.20	122.06	1	7	10	6.00
PolBoycott	5.96	4.14	84.77	6	9	17	10.67
FutureRespect	3.89	2.95	94.04	8	15	16	13.00
ICQThrift	3.82	4.27	63.15	9	6	24	13.00
ACTReligion	0.92	3.83	94.97	19	10	15	14.67
ICQTolerance	3.94	3.00	61.43	7	13	25	15.00
Competition	0.13	3.27	206.94	30	12	4	15.33
ILLeisure	0.94	2.32	123.03	17	20	9	15.33
ILWork	2.48	2.38	97.51	13	19	14	15.33
Trusted	3.02	3.46	59.32	10	11	27	16.00
PolExperts	0.73	1.59	145.44	20	25	6	17.00
ILFriends	0.92	2.13	109.49	18	22	12	17.33
Happy	0.13	2.54	115.79	29	18	11	19.33
ACTArtsEd	0.36	2.80	68.20	23	17	19	19.67
FutureWork	0.32	1.98	108.16	24	23	13	20.00
ICQHardWork	1.66	1.19	67.11	15	28	20	21.00
ICQObedience	2.16	1.69	60.41	14	24	26	21.33
ICQDetermination	0.57	1.47	70.46	21	26	18	21.67
ACTEnvOrg	1.26	2.17	49.96	16	21	30	22.33
ACTProfessional	0.17	2.95	57.95	28	14	28	23.33
ICQUnselfishness	0.55	1.26	64.65	22	27	23	24.00
ILFam	0.20	2.87	38.45	27	16	31	24.67
ICQResponsibility	0.28	0.28	65.59	25	31	21	25.67
Sex	0.20	0.69	64.82	26	29	22	25.67
ICQImagination	0.12	0.57	56.63	31	30	29	30.00

Wave was the most important overall predictor across the ensemble models. This suggests possible societal changes in attitudes toward television.

Most important overall

Across targets: religious variables (ILReligion, ACTReligion) dominate CChurches; political engagement (PolInterest, ILPolitics) dominates CParliament/CTelevision; Age, HardWork, PrivateState are consistently strong secondary predictors.

Least important overall

Variables such as ICQImagination, Sex, and ICQResponsibility ranked consistently low across all models, meaning no model found them useful for classification tasks and removing them could be reasonable.

Q9: Explaining Performance Differences Across Classification Models

The performance differences reflect how each model's mechanism interacts with the WVS data's characteristics - data missingness and varying signal strength across targets. Ensemble methods (RF, Boosting) consistently dominated AUC, consistent with Kyriazos & Poga (2024), who show that ensembles better capture the non-linear relationships typical of social-science data, and with Pencheva (2025) on the robustness of aggregated learners under noisy self-report data. Decision Tree's collapse on CTelevision (AUC = 0.500) illustrates that when no single feature provides sufficient information gain.

Q10: Improving the Worst Performing Ensemble Model

Bagging is consistently the weakest ensemble (Avg F1 = 0.440, Avg AUC = 0.644, both lowest), and is therefore selected for improvement.

Q10: Average F1-score Across Ensemble Classifiers

Classifier	CChurches	CTelevision	CParliament	Average_F1
Bagging	0.810	0.301	0.210	0.440
Boosting	0.815	0.501	0.333	0.550
Random Forest	0.816	0.473	0.291	0.526

Q10: Average AUC Across Ensemble Classifiers

Classifier	CChurches	CTelevision	CParliament	Avg_AUC
Bagging	0.728	0.599	0.606	0.644
Boosting	0.794	0.620	0.643	0.686
Random Forest	0.792	0.622	0.648	0.687

Key decision factors: feature subset size (20), parameter tuning, and 5-fold CV.

- (1) **Feature selection:** top 20 predictors by average importance rank across the three ensembles per target, mitigating single-model bias. This helps bagging to reduce noise
- (2) **Parameter tuning:** $m_{final} \in \{50, 100, 150, 200, 300\}$ to find out which m_{final} have the greatest variance reduction.
- (3) **5-fold CV on F1:** performed on the training set. optimal m_{final} was 50 (CChurches, CTelevision), 200 (CParliament), then used to retrain on full training data before test evaluation.

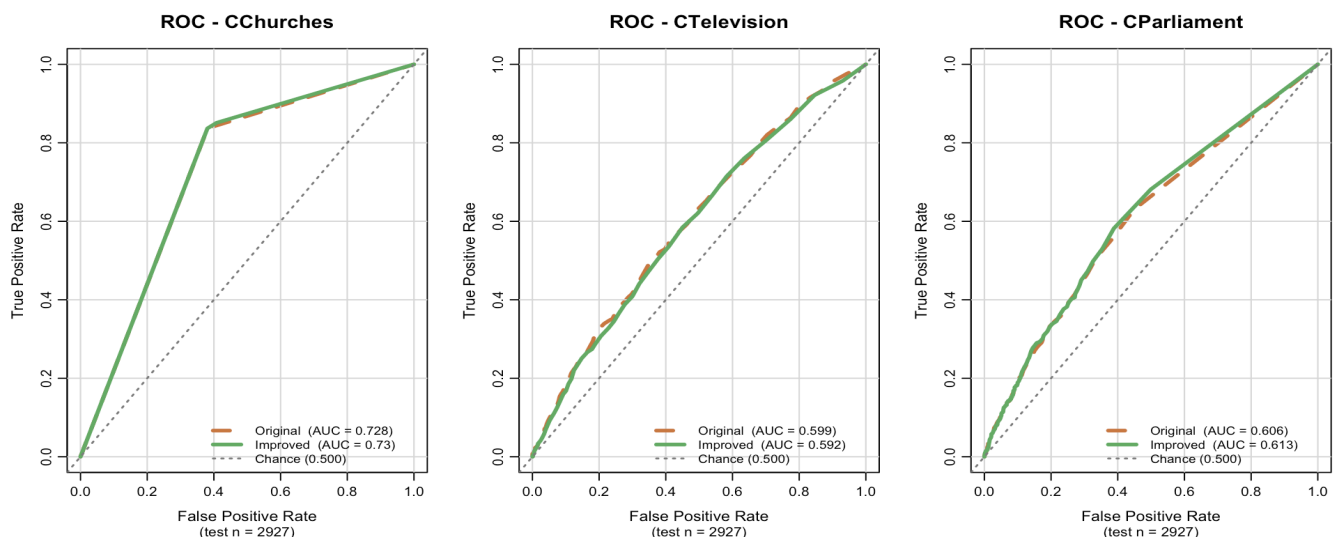
F1 Comparison: Original Bagging vs Improved Bagging

Target	F1_Original	F1_Improved	Delta_F1	Relative_Gain_pct
CChurches	0.810	0.810	0.000	0.00
CTelevision	0.301	0.297	-0.004	-1.46
CParliament	0.210	0.226	0.016	7.58

AUC Comparison: Original Bagging vs Improved Bagging

Target	AUC_Original	AUC_Improved	Delta_AUC
CChurches	0.728	0.730	0.002
CTelevision	0.599	0.592	-0.007
CParliament	0.606	0.613	0.007

ROC Curves: Original vs Improved Bagging

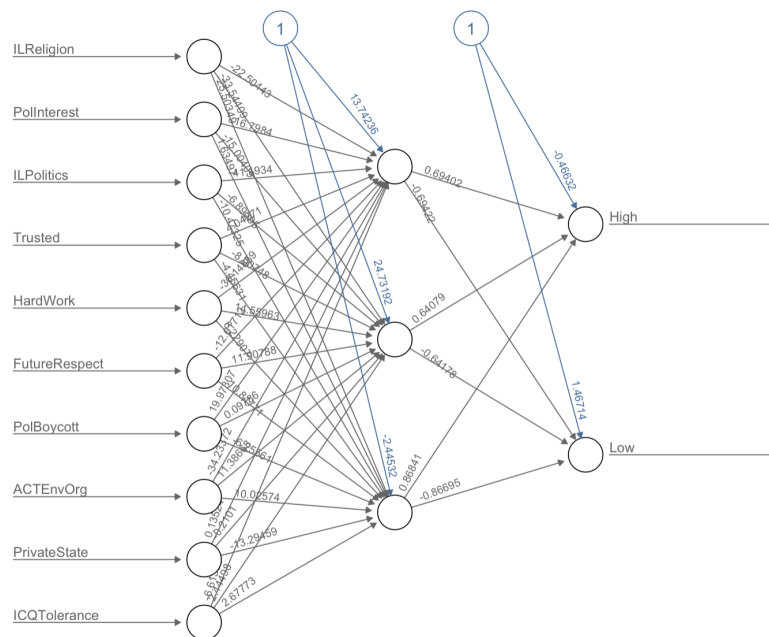


On CChurches, the dominant ILReligion is already being captured by all ensembles, no F1 improvement was observed.

On CTelevision, the negative number shows that even tuned Bagging cannot extract more insights. Poor performance may reflect weak predictor signal and information loss due to heavy missingness of data which may also affect CTelevision.

The largest improvement observed on CParliament ($\Delta F1 = +0.016$ and $\Delta AUC = +0.007$), more trees ($m_{final} = 200$) did improve discrimination ability.

Q11: ANN Classifier for USA: CChurches Prediction



Country selection. USA had the greatest number of training observations (n = 264) based on cleaned data.

Target variable. CChurches.

Predictors (10): top 10 by average importance rank across the three Q8 ensembles (averaged over targets). Country and Wave excluded. Top 3 capture ~70% importance; 10 retained to let the ANN exploit non-linear interactions.

The selected predictors span three categories:

- Religious/membership attitudes (ILReligion, ACTEnvOrg)
- Political engagement (PolInterest, ILPolitics, PolBoycott)
- Personal values (Trusted, HardWork, FutureRespect, PrivateState)
- Important child qualities (ICQTolerance)

Data pre-processing.

1. **Factor to numeric conversion:** preserving original 0/1 (or ordinal) codes.
2. **Min-max normalisation to [0,1]** applied to all predictors (ANN is sensitive to input scale).
3. **Binary indicator outputs:** two indicator columns were created (High = CChurches == 1, Low = CChurches == 0).

Architecture: 10 inputs; 1 hidden layer * 3 nodes; 2 outputs (High/Low); threshold = 0.05, stepmax = 1e6.

Training adjusted weights in ~2250 steps with final error ~30. A deeper alternative hidden = c(8, 4) was tested but produced lower test accuracy, thus the simpler architecture was retained.

Q12: Wave-Based Evaluation of ANN Classification Performance

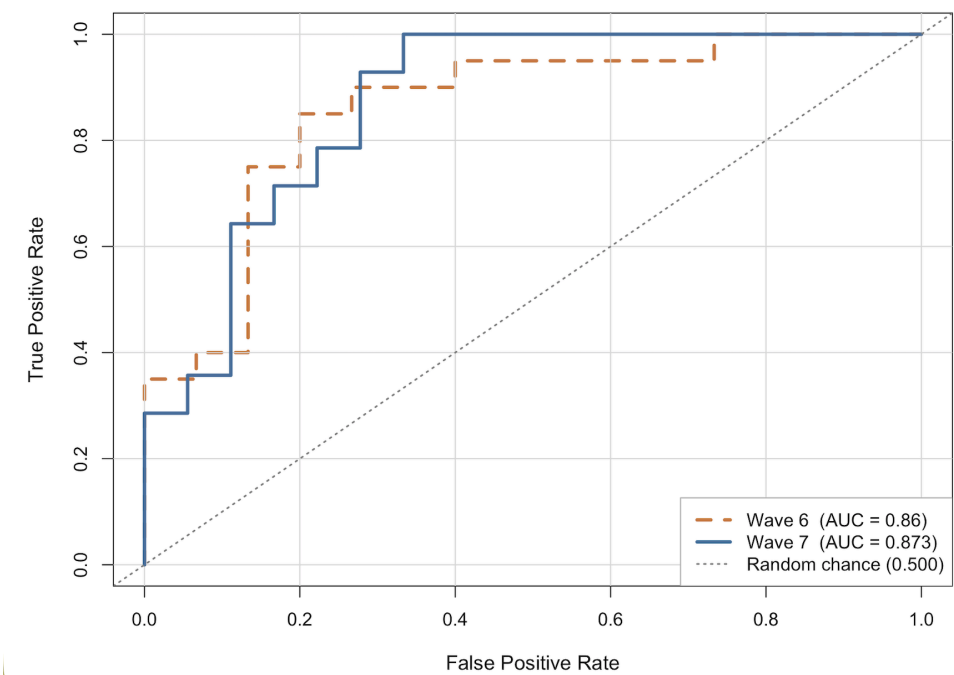
Wave 6 (n = 35) and Wave 7 (n = 32) have the greatest USA observations. Across the two waves with the most USA observations, the model maintained high, even slightly increasing from 0.860 (Wave 6) to 0.873 (Wave 7). These numbers are closer to 1 - perfect model, indicating that its underlying ability to rank 'High-confidence' instances above 'Low' was preserved over time. F1-score dropped from 0.789 (Wave 6) to 0.714 (Wave 7), reflecting a small loss in classification, primarily due to a drop in precision (0.833 to 0.714). The variation in F1 could be reflected by the small sample size as small changes in classification could significantly change F1.

Both ROC curves bow well above the diagonal and follow similar shapes, confirming consistent discriminative behaviour across waves. A subtle crossover is visible: Wave 6's curve dominates at low FPR (< 0.15), while Wave 7's curve overtakes at higher FPR, reaching TPR = 1.0 earlier.

ANN performance across two waves (USA)

Wave	N	Accuracy	Precision	Recall	F1	AUC
6	35	0.771	0.833	0.750	0.789	0.860
7	32	0.750	0.714	0.714	0.714	0.873

ROC Curves: ANN performance across two waves (USA)



References

- Kyriazos, T., & Poga, M. (2024). *Application of machine learning models in social sciences: Managing nonlinear relationships*. *Encyclopedia*, 4(4), 1790–1805. <https://doi.org/10.3390/encyclopedia4040118>
- Pencheva, D. (2025). *Profiling noisy social media data for sentiment applications: A visual and analytical framework*. *SAR Journal*, 8(3), 213–224. <https://doi.org/10.18421/SAR83-01>

Appendix

```
#set up
rm(list = ls())
set.seed(33235422) # Your Student ID is the random seed
WD = read.csv("WVSBinaryExtract.csv")
selected_cols = c(sample(3:49, 30), sample(50:63, 3))
WD = WD[c(1:2, selected_cols)]
WD = WD[sample(nrow(WD), 20000, replace = FALSE),]

library(tree)
library(e1071)
library(adabag)
library(randomForest)
library(ROCR)
library(pROC)
library(caret)
library(ggplot2)
library(dplyr)
library(knitr)
library(gridExtra)
library(grid)
library(kableExtra)
library(fields)
library(webshot2)
library(tidyr)

#Q1 data exploration

#Overall dataset
str(WD)
length(unique(WD$Country))
sort(table(WD$Country), decreasing = TRUE)[1:10]
sort(table(WD$Wave), decreasing = TRUE)
summary(WD)
all_vars <- names(WD)

il_vars      <- grep("^IL",      all_vars, value = TRUE)
icq_vars     <- grep("^ICQ",     all_vars, value = TRUE)
act_vars     <- grep("^ACT",     all_vars, value = TRUE)
pol_vars     <- grep("^Pol",     all_vars, value = TRUE)
future_vars  <- grep("^Future",  all_vars, value = TRUE)
conf_vars    <- c("CChurches", "CTelevision", "CParliament")

cat("IL*:    "); print(il_vars)
cat("ICQ*:   "); print(icq_vars)
cat("ACT*:   "); print(act_vars)
cat("Pol*:   "); print(pol_vars)
cat("Future*"); print(future_vars)

# min / max / unique values
check_range <- function(vars, df = WD) {
  for (v in vars) {
    x <- df[[v]]
    x_clean <- x[x >= 0 & !is.na(x)]
    cat(v, ":",
        "range =", min(x_clean), "-", max(x_clean),
        "| unique =", paste(sort(unique(x_clean)), collapse = ","),
        "\n")
  }
}
```

```

    }
}

cat("\n=== IL* (Importance in life) ===\n")
check_range(il_vars)

cat("\n=== ICQ* (Important Child Qualities) ===\n")
check_range(icq_vars)

cat("\n=== ACT* (Civic Activity) ===\n")
check_range(act_vars)

cat("\n=== Pol* (Political Attitudes) ===\n")
check_range(pol_vars)

cat("\n=== Future* ===\n")
check_range(future_vars)

# ----age
age_clean <- WD$Age[WD$Age > 0 & !is.na(WD$Age)]

# histogram
hist(age_clean,
      breaks = 30,
      col     = "#2F6F9F",
      border  = "white",
      xlab    = "Age (years)",
      ylab    = "Frequency",
      main    = "Age Distribution")

abline(v = median(age_clean), col = "#E07A3F", lwd = 2, lty = 2)
abline(v = mean(age_clean), col = "#27AE60", lwd = 2, lty = 2)

legend("topright",
      legend = c(paste0("Median = ", median(age_clean)),
                  paste0("Mean = ", round(mean(age_clean), 1))),
      col     = c("#E07A3F", "#27AE60"),
      lty     = 2, lwd = 2, cex = 0.9, bty = "n")

# sex
sex_clean <- WD$Sex[WD$Sex > 0 & !is.na(WD$Sex)]

sex_count <- table(sex_clean)
print(sex_count)

sex_prop <- prop.table(sex_count)
print(round(sex_prop, 4))

cat("\nSex distribution: Male =", sex_count[1],
    sprintf("%.1f%%", sex_prop[1]*100),
    "| Female =", sex_count[2],
    sprintf("%.1f%%", sex_prop[2]*100), "\n")

cat("Missing Sex (negative codes + NA):",
    sum(WD$Sex <= 0 | is.na(WD$Sex)), "\n")
#high/low proportion for 3 confidence variables
confidence_vars <- c("CChurches", "CTelevision", "CParliament")

#calculate count, proportion, H:L to a list

```

```

conf_stats <- lapply(confidence_vars, function(v) {
  count <- table(WD[[v]])
  print(count)
  list(count = count,
        prop = prop.table(count))
})

names(conf_stats) <- confidence_vars
conf_stats

# Confidence proportions plot (saved to Q1_confidence_proportions.png)
png("Q1_confidence_proportions.png",
    width = 1500, height = 750, res = 150)

par(mfrow = c(1, 3),
    oma = c(0, 0, 3, 0),
    mar = c(4, 4, 3, 1))

for (v in confidence_vars) {
  count <- table(WD[[v]])
  prop <- prop.table(count)
  bp <- barplot(count,
    names.arg = c("Low (0)", "High (1)"),
    col = c("#E07A3F", "#2F6F9F"),
    ylim = c(0, max(count) * 1.3),
    ylab = "Count",
    main = paste("Confidence:", sub("^C", "", v)))
  text(bp, count,
    labels = paste0(format(count, big.mark = ","),
      "\n(", round(prop * 100, 1), "%)"),
    pos = 3, cex = 0.8, font = 2)
}

mtext("3 Confidence Variables Proportions\n(Confidence = 1 High vs. 0 Low)",
    outer = TRUE, cex = 1.2, font = 2)

dev.off()

par(mfrow = c(1, 1))

#data quality checking
num_cols <- sapply(WD, is.numeric)
miss_pct <- sapply(WD[, num_cols], function(x) {
  mean(x < 0 | is.na(x)) * 100
})

miss_pct <- sort(miss_pct)

bar_colors <- ifelse(miss_pct < 5, "#3CB371",
  ifelse(miss_pct <= 20, "orange",
    "red"))

# === Data quality plot
png("fig2_missing_values.png",
    width = 1500, height = 900, res = 150)

par(mar = c(4, 9, 4, 2))

bp <- barplot(miss_pct,
  horiz = TRUE,

```

```

        las= 1,
        col = bar_colors,
        border= NA,
        xlim= c(0, max(miss_pct) * 1.15),
        xlab = "Missing / Negative-code Percentage (%)",
        main= "Data Quality: Missing Response Rate by Attribute",
        cex.names = 0.7)

text(miss_pct, bp,
     labels = paste0(round(miss_pct, 1), "%"),
     pos = 4, cex = 0.7)

# threshold lines
abline(v = 5, col = "#3CB371", lty = 2)
abline(v = 20, col = "#E89B3B", lty = 2)

legend("bottomright",
      legend = c("<5% (Good)", "5-20% (Moderate)", ">20% (High)"),
      fill = c("#3CB371", "#E89B3B", "#E55A4F"),
      bty = "o", cex = 0.85)

dev.off()
#country & wave
#country
length(unique(WD$Country))
sort(table(WD$Country), decreasing = TRUE)[1:10]

# wave
table(WD$Wave)
sort(table(WD$Wave), decreasing = TRUE)[1:7]

# wave count across countries
table(WD$Country, WD$Wave)

#count na in different conf variable
sum(is.na(WD$CChurches))
sum(is.na(WD$CTelevision))
sum(is.na(WD$CParliament))

#Q2
WD_Clean <- WD
summary(WD_Clean)

#convert all negative number to na
WD_Clean[num_cols] <- lapply(WD[num_cols], function(x) ifelse(x < 0, NA, x))
WD_Clean <- WD_Clean[complete.cases(WD_Clean), ]

#convert to factor
WD_Clean$CChurches<- as.factor(WD_Clean$CChurches)
WD_Clean$CTelevision<- as.factor(WD_Clean$CTelevision)
WD_Clean$CParliament<- as.factor(WD_Clean$CParliament)
WD_Clean$Country<- as.factor(WD_Clean$Country)
WD_Clean$Wave <- as.factor(WD_Clean$Wave)
WD_Clean$Sex <- as.factor(WD_Clean$Sex)

write.csv(WD_Clean, "WD_Clean.csv", row.names = FALSE)

# check wd and wd clean before and after
cat("Original:", nrow(WD), "rows\n")
cat("Cleaned :", nrow(WD_Clean), "rows\n")
cat("Lost :", round((1 - nrow(WD_Clean)/nrow(WD)) * 100, 1), "%\n")

```

```

#-----Q3
set.seed(33235422)
train.row <- sample(1:nrow(WD_Clean), 0.7 * nrow(WD_Clean))
WD.train <- WD_Clean[train.row, ]
WD.test <- WD_Clean[-train.row, ]

write.csv(WD.train, "WD_train.csv", row.names = FALSE)
write.csv(WD.test, "WD_test.csv", row.names = FALSE)

cat("Total rows:", nrow(WD_Clean), "\n")
cat("Train rows:", nrow(WD.train), "(",
round(nrow(WD.train)/nrow(WD_Clean)*100, 1), "%)\n")
cat("Test rows :", nrow(WD.test), "(", round(nrow(WD.test)/nrow(WD_Clean)*100,
1), "%)\n")

#-----Q4
#CChurches
churches_models <- list()

tree_churches <- tree(CChurches ~ . - CTelevision - CParliament - Country, data
= WD.train)
nb_churches <- naiveBayes(CChurches ~ . - CTelevision - CParliament - Country,
data = WD.train)
bag_churches <- bagging(CChurches ~ . - CTelevision - CParliament - Country,
data = WD.train)
boost_churches <- boosting(CChurches ~ . - CTelevision - CParliament - Country,
data = WD.train)
rf_churches <- randomForest(CChurches ~ . - CTelevision - CParliament -
Country, data = WD.train)

churches_models$tree <- tree_churches
churches_models$nb <- nb_churches
churches_models$bag <- bag_churches
churches_models$boost <- boost_churches
churches_models$rf <- rf_churches

#CTelevision
television_models <- list()

television_models$tree <- tree(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$nb <- naiveBayes(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$bag <- bagging(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$boost <- boosting(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$rf <- randomForest(CTelevision ~ . - CChurches - CParliament
- Country, data = WD.train)

#CParliament
parliament_models <- list()

parliament_models$tree <- tree(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$nb <- naiveBayes(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)

```



```

parliament_models$bag <- bagging(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$boost <- boosting(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$rfr <- randomForest(CParliament ~ . - CChurches - CTelevision
- Country, data = WD.train)

#---Q5
#--Predict CChurches using test data
#dc
Chur_pred_tree <- predict(churches_models$tree,WD.test, type = "class")
Chur_t1 = table(Predicted_Class = Chur_pred_tree, Actual_Class =
WD.test$CChurches)
cat("\n#Decision Tree Confusion\n")
print(Chur_t1)

      paste0("Mean = ", round(mean(age_clean), 1))),
col      = c("#E07A3F", "#27AE60"),
lty      = 2, lwd = 2, cex = 0.9, bty = "n")

# sex
sex_clean <- WD$Sex[WD$Sex > 0 & !is.na(WD$Sex)]

sex_count <- table(sex_clean)
print(sex_count)

sex_prop <- prop.table(sex_count)
print(round(sex_prop, 4))

cat("\nSex distribution: Male =", sex_count[1],
sprintf("(%.1f%%)", sex_prop[1]*100),
"| Female =", sex_count[2],
sprintf("(%.1f%%)", sex_prop[2]*100), "\n")

cat("Missing Sex (negative codes + NA):",
sum(WD$Sex <= 0 | is.na(WD$Sex)), "\n")
#high/low proportion for 3 confidence variables
confidence_vars <- c("CChurches", "CTelevision", "CParliament")

#calculate count, proportion, H:L to a list
conf_stats <- lapply(confidence_vars, function(v) {
  count <- table(WD[[v]])
  print(count)
  list(count = count,
       prop = prop.table(count))
})

names(conf_stats) <- confidence_vars
conf_stats

# Confidence proportions plot (saved to Q1_confidence_proportions.png)
png("Q1_confidence_proportions.png",
    width = 1500, height = 750, res = 150)

par(mfrow = c(1, 3),
    oma = c(0, 0, 3, 0),
    mar = c(4, 4, 3, 1))

for (v in confidence_vars) {

```

```

count <- table(WD[[v]])
prop <- prop.table(count)
bp <- barplot(count,
              names.arg = c("Low (0)", "High (1)"),
              col = c("#E07A3F", "#2F6F9F"),
              ylim = c(0, max(count) * 1.3),
              ylab = "Count",
              main = paste("Confidence:", sub("^C", "", v)))
text(bp, count,
     labels = paste0(format(count, big.mark = ","),
                     "\n(", round(prop * 100, 1), "%)"),
     pos = 3, cex = 0.8, font = 2)
}

mtext("3 Confidence Variables Proportions\n(Confidence = 1 High vs. 0 Low)",
     outer = TRUE, cex = 1.2, font = 2)

dev.off()

par(mfrow = c(1, 1))

#data quality checking
num_cols <- sapply(WD, is.numeric)
miss_pct <- sapply(WD[, num_cols], function(x) {
  mean(x < 0 | is.na(x)) * 100
})

miss_pct <- sort(miss_pct)

bar_colors <- ifelse(miss_pct < 5, "#3CB371",
                    ifelse(miss_pct <= 20, "orange",
                          "red"))

# === Data quality plot (saved to fig2_missing_values.png)
png("fig2_missing_values.png",
    width = 1500, height = 900, res = 150)

par(mar = c(4, 9, 4, 2))

bp <- barplot(miss_pct,
              horiz = TRUE,
              las = 1,
              col = bar_colors,
              border = NA,
              xlim = c(0, max(miss_pct) * 1.15),
              xlab = "Missing / Negative-code Percentage (%)",
              main = "Data Quality: Missing Response Rate by Attribute",
              cex.names = 0.7)

text(miss_pct, bp,
     labels = paste0(round(miss_pct, 1), "%"),
     pos = 4, cex = 0.7)

# threshold lines
abline(v = 5, col = "#3CB371", lty = 2)
abline(v = 20, col = "#E89B3B", lty = 2)

legend("bottomright",
     legend = c("<5% (Good)", "5-20% (Moderate)", ">20% (High)"),
     fill = c("#3CB371", "#E89B3B", "#E55A4F"),

```

```

    bty      = "o", cex = 0.85)

dev.off()
#country & wave
#country
length(unique(WD$Country))
sort(table(WD$Country), decreasing = TRUE)[1:10]

# wave
table(WD$Wave)
sort(table(WD$Wave), decreasing = TRUE)[1:7]

# wave count across countries
table(WD$Country, WD$Wave)

#count na in different conf variable
sum(is.na(WD$CChurches))
sum(is.na(WD$CTelevision))
sum(is.na(WD$CParliament))

#Q2
WD_Clean <- WD
summary(WD_Clean)

#convert all negative number to na
WD_Clean[num_cols] <- lapply(WD[num_cols], function(x) ifelse(x < 0, NA, x))
WD_Clean <- WD_Clean[complete.cases(WD_Clean), ]

#convert to factor
WD_Clean$CChurches<- as.factor(WD_Clean$CChurches)
WD_Clean$CTelevision<- as.factor(WD_Clean$CTelevision)
WD_Clean$CParliament<- as.factor(WD_Clean$CParliament)
WD_Clean$Country<- as.factor(WD_Clean$Country)
WD_Clean$Wave <- as.factor(WD_Clean$Wave)
WD_Clean$Sex <- as.factor(WD_Clean$Sex)

write.csv(WD_Clean, "WD_Clean.csv", row.names = FALSE)

# check wd and wd clean before and after
cat("Original:", nrow(WD), "rows\n")
cat("Cleaned :", nrow(WD_Clean), "rows\n")
cat("Lost :", round((1 - nrow(WD_Clean)/nrow(WD)) * 100, 1), "%\n")

#-----Q3
set.seed(33235422)
train.row <- sample(1:nrow(WD_Clean), 0.7 * nrow(WD_Clean))
WD.train <- WD_Clean[train.row, ]
WD.test <- WD_Clean[-train.row, ]

write.csv(WD.train, "WD_train.csv", row.names = FALSE)
write.csv(WD.test, "WD_test.csv", row.names = FALSE)

cat("Total rows:", nrow(WD_Clean), "\n")
cat("Train rows:", nrow(WD.train), "(",
round(nrow(WD.train)/nrow(WD_Clean)*100, 1), "%)\n")
cat("Test rows :", nrow(WD.test), "(", round(nrow(WD.test)/nrow(WD_Clean)*100,
1), "%)\n")

```

```

#-----Q4
#CChurches
churches_models <- list()

tree_churches <- tree(CChurches ~ . - CTelevision - CParliament - Country, data
= WD.train)
nb_churches <- naiveBayes(CChurches ~ . - CTelevision - CParliament - Country,
data = WD.train)
bag_churches <- bagging(CChurches ~ . - CTelevision - CParliament - Country,
data = WD.train)
boost_churches <- boosting(CChurches ~ . - CTelevision - CParliament - Country,
data = WD.train)
rf_churches <- randomForest(CChurches ~ . - CTelevision - CParliament -
Country, data = WD.train)

churches_models$tree <- tree_churches
churches_models$nb <- nb_churches
churches_models$bag <- bag_churches
churches_models$boost <- boost_churches
churches_models$rf <- rf_churches

#CTelevision
television_models <- list()

television_models$tree <- tree(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$nb <- naiveBayes(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$bag <- bagging(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$boost <- boosting(CTelevision ~ . - CChurches - CParliament -
Country, data = WD.train)
television_models$rf <- randomForest(CTelevision ~ . - CChurches - CParliament
- Country, data = WD.train)
print(television_models$tree)
#CParliament
parliament_models <- list()

parliament_models$tree <- tree(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$nb <- naiveBayes(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$bag <- bagging(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$boost <- boosting(CParliament ~ . - CChurches - CTelevision -
Country, data = WD.train)
parliament_models$rf <- randomForest(CParliament ~ . - CChurches - CTelevision
- Country, data = WD.train)

print(parliament_models$tree)
#---Q5
#--Predict CChurches using test data
#dc

print(churches_models$tree)
Chur_pred_tree <- predict(churches_models$tree,WD.test, type = "class")
Chur_t1 = table(Predicted_Class = Chur_pred_tree, Actual_Class =
WD.test$CChurches)
cat("\n#Decision Tree Confusion\n")
print(Chur_t1)

```

```

#nb
Chur_pred_nb <- predict(churches_models$nb, WD.test)
Chur_t2 = table(Predicted_Class = Chur_pred_nb, Actual_Class =
WD.test$CChurches)
cat("\n#NaiveBayes Confusion\n")
print(Chur_t2)

#bagging
print(churches_models$bag$importance)
chur_pred_bag <- predict.bagging(churches_models$bag, WD.test)
Chur_t3 <- table(
  Predicted_Class = chur_pred_bag$class,
  Actual_Class = WD.test$CChurches
)
cat("\n#Bagging Confusion\n")
print(Chur_t3)

#boosting
chur_pred_boost <- predict.boosting(churches_models$boost, WD.test)
Chur_t4 <- table(
  Predicted_Class = chur_pred_boost$class,
  Actual_Class = WD.test$CChurches
)
cat("\n#Boosting Confusion\n")

print(Chur_t4)

chur_pred_rf <- predict(churches_models$rf, WD.test, na.action = na.exclude)
Chur_t5 <- table(
  Predicted_Class = chur_pred_rf,
  Actual_Class = WD.test$CChurches
)
cat("\n#random forest Confusion\n")
print(Chur_t5)

#--Predict CTelevision using test data
tel_pred_tree <- predict(television_models$tree,WD.test, type = "class")
Tel_t1 = table(Predicted_Class = tel_pred_tree, Actual_Class =
WD.test$CTelevision)
cat("\n#Decision Tree Confusion\n")
print(Tel_t1)

tel_pred_nb <- predict(television_models$nb, WD.test)
Tel_t2 = table(Predicted_Class = tel_pred_nb, Actual_Class =
WD.test$CTelevision)
cat("\n#NaiveBayes Confusion\n")
print(Tel_t2)

tel_pred_bag <- predict.bagging(television_models$bag, WD.test)
Tel_t3 <- table(Predicted_Class = tel_pred_bag$class,
  Actual_Class = WD.test$CTelevision)
cat("\n#Bagging Confusion\n")
print(Tel_t3)

tel_pred_boost <- predict.boosting(television_models$boost, WD.test)
Tel_t4 <- table(Predicted_Class = tel_pred_boost$class, Actual_Class =
WD.test$CTelevision)

```

```

cat("\n#Boosting Confusion\n")
print(Tel_t4)

tel_pred_rf <- predict(television_models$rf, WD.test, na.action = na.exclude)
Tel_t5 <- table(Predicted_Class = tel_pred_rf,
                Actual_Class = WD.test$CTelevision)
cat("\n#Random Forest Confusion\n")
print(Tel_t5)

#nb
Chur_pred_nb <- predict(churches_models$nb, WD.test)
Chur_t2 = table(Predicted_Class = Chur_pred_nb, Actual_Class =
WD.test$CChurches)
cat("\n#NaiveBayes Confusion\n")
print(Chur_t2)

#bagging
chur_pred_bag <- predict.bagging(churches_models$bag, WD.test)
Chur_t3 <- table(
  Predicted_Class = chur_pred_bag$class,
  Actual_Class = WD.test$CChurches
)
cat("\n#Bagging Confusion\n")
print(Chur_t3)

#boosting
chur_pred_boost <- predict.boosting(churches_models$boost, WD.test)
Chur_t4 <- table(
  Predicted_Class = chur_pred_boost$class,
  Actual_Class = WD.test$CChurches
)
cat("\n#Boosting Confusion\n")

print(Chur_t4)

chur_pred_rf <- predict(churches_models$rf, WD.test, na.action = na.exclude)
Chur_t5 <- table(
  Predicted_Class = chur_pred_rf,
  Actual_Class = WD.test$CChurches
)
cat("\n#random forest Confusion\n")
print(Chur_t5)

#--Predict CTelevision using test data
tel_pred_tree <- predict(television_models$tree,WD.test, type = "class")
Tel_t1 = table(Predicted_Class = tel_pred_tree, Actual_Class =
WD.test$CTelevision)
cat("\n#Decision Tree Confusion\n")
print(Tel_t1)

tel_pred_nb <- predict(television_models$nb, WD.test)
Tel_t2 = table(Predicted_Class = tel_pred_nb, Actual_Class =
WD.test$CTelevision)
cat("\n#NaiveBayes Confusion\n")
print(Tel_t2)

```

```

tel_pred_bag <- predict.bagging(television_models$bag, WD.test)
Tel_t3 <- table(Predicted_Class = tel_pred_bag$class,
               Actual_Class = WD.test$CTelevision)
cat("\n#Bagging Confusion\n")
print(Tel_t3)

tel_pred_boost <- predict.boosting(television_models$boost, WD.test)
Tel_t4 <- table(Predicted_Class = tel_pred_boost$class, Actual_Class =
WD.test$CTelevision)
cat("\n#Boosting Confusion\n")
print(Tel_t4)

tel_pred_rf <- predict(television_models$rf, WD.test, na.action = na.exclude)
Tel_t5 <- table(Predicted_Class = tel_pred_rf,
               Actual_Class = WD.test$CTelevision)
cat("\n#Random Forest Confusion\n")
print(Tel_t5)

#--Predict CParliament using test data
# Decision Tree

par_pred_tree <- predict(parliament_models$tree, WD.test, type = "class")
Par_t1 <- table(Predicted_Class = par_pred_tree, Actual_Class =
WD.test$CParliament)
cat("\n#Decision Tree Confusion\n")
print(Par_t1)

# Naive Bayes
par_pred_nb <- predict(parliament_models$nb, WD.test)
Par_t2 <- table(Predicted_Class = par_pred_nb, Actual_Class =
WD.test$CParliament)
cat("\n#NaiveBayes Confusion\n")
print(Par_t2)

# Bagging
par_pred_bag <- predict.bagging(parliament_models$bag, WD.test)
Par_t3 <- table(Predicted_Class = par_pred_bag$class, Actual_Class =
WD.test$CParliament)
cat("\n#Bagging Confusion\n")
print(Par_t3)

# Boosting
par_pred_boost <- predict.boosting(parliament_models$boost, WD.test)
Par_t4 <- table(Predicted_Class = par_pred_boost$class, Actual_Class =
WD.test$CParliament)
cat("\n#Boosting Confusion\n")
print(Par_t4)

# Random Forest
par_pred_rf <- predict(parliament_models$rf, WD.test, na.action = na.exclude)
Par_t5 <- table(Predicted_Class = par_pred_rf, Actual_Class =
WD.test$CParliament)

cat("\n#Random Forest Confusion\n")
print(Par_t5)

#---- Q5 visualization: grouped bar chart per target

```

```

#--helper to get metrics
get_metrics <- function(cm)
{

  TN <- cm["0", "0"]
  FP <- cm["1", "0"]
  FN <- cm["0", "1"]
  TP <- cm["1", "1"]

  accuracy<- (TP + TN) / sum(cm)
  precision <- ifelse(
    (TP + FP) == 0,
    0,
    TP / (TP + FP)
  )

  recall <- ifelse(
    (TP + FN) == 0,
    0,
    TP / (TP + FN)
  )

  f1 <- ifelse(
    (precision + recall) == 0,
    0,
    2 * precision * recall / (precision + recall)
  )

  round(c(Accuracy = accuracy,
          Precision = precision,
          Recall= recall,
          F1= f1), 4)
}

# CChurches - use Chur_t1...Chur_t5
Chur_metrics <- rbind(
  Tree = get_metrics(Chur_t1),
  NaiveBayes = get_metrics(Chur_t2),
  Bagging = get_metrics(Chur_t3),
  Boosting = get_metrics(Chur_t4),
  RandomForest = get_metrics(Chur_t5)
)
cat("\n### CChurches Metrics\n")
print(Chur_metrics)

# CTelevision - use Tel_t1...Tel_t5
Tel_metrics <- rbind(
  Tree = get_metrics(Tel_t1),
  NaiveBayes= get_metrics(Tel_t2),
  Bagging = get_metrics(Tel_t3),
  Boosting = get_metrics(Tel_t4),
  RandomForest = get_metrics(Tel_t5)
)

cat("\n### CTelevision Metrics\n")
print(Tel_metrics)

# CParliament metrics

```



```

Par_metrics <- rbind(
  Tree = get_metrics(Par_t1),
  NaiveBayes = get_metrics(Par_t2),
  Bagging = get_metrics(Par_t3),
  Boosting = get_metrics(Par_t4),
  RandomForest = get_metrics(Par_t5)
)

cat("\n### CParliament Metrics\n")
print(Par_metrics)
print(Par_t1)

#best f1
best_row_chur_f1 <- which.max(Chur_table$F1)
best_row_par_f1 <- which.max(Par_table$F1)
best_row_tel_f1 <- which.max(Tel_table$F1)

#----- CChurches -----
Chur_table <- data.frame(
  Model = rownames(Chur_metrics),
  Accuracy = Chur_metrics[, "Accuracy"],
  Precision = Chur_metrics[, "Precision"],
  Recall = Chur_metrics[, "Recall"],
  F1 = Chur_metrics[, "F1"]
)

rownames(Chur_table) <- NULL

kable(Chur_table, digits = 3) %>%
  kable_styling(full_width = TRUE) %>%
  row_spec(0, background = "#D9E2F3") %>%
  row_spec(best_row_chur_f1, background = "#FFF2CC")

#----- CTelevision -----
Tel_table <- data.frame(
  Model = rownames(Tel_metrics),
  Accuracy = Tel_metrics[, "Accuracy"],
  Precision = Tel_metrics[, "Precision"],
  Recall = Tel_metrics[, "Recall"],
  F1 = Tel_metrics[, "F1"]
)

rownames(Tel_table) <- NULL

kable(Tel_table, digits = 3) %>%
  kable_styling(full_width = TRUE) %>%
  row_spec(0, background = "#D9E2F3") %>%
  row_spec(best_row_tel_f1, background = "#FFF2CC")

#----- CParliament -----
Par_table <- data.frame(
  Model = rownames(Par_metrics),
  Accuracy = Par_metrics[, "Accuracy"],
  Precision = Par_metrics[, "Precision"],
  Recall = Par_metrics[, "Recall"],
  F1 = Par_metrics[, "F1"]
)

```

```

rownames(Par_table) <- NULL

kable(Par_table, digits = 3) %>%
  kable_styling(full_width = TRUE) %>%
  row_spec(0, background = "#D9E2F3") %>%
  row_spec(best_row_par_f1, background = "#FFF2CC")

#confusion matrix
plot_cm_bw <- function(cm, title, show_y = TRUE) {
  cm_t <- t(cm)
  df <- as.data.frame(as.table(cm_t))
  colnames(df) <- c("True", "Pred", "Count")
  df$True <- factor(df$True, levels = c("1", "0"))
  df$Pred <- factor(df$Pred, levels = c("0", "1"))

  p <- ggplot(df, aes(x = Pred, y = True)) +
    geom_tile(fill = "white", color = "black", linewidth = 0.6) +
    geom_text(aes(label = format(Count, big.mark = ",")),
              fontface = "bold", size = 4.5) +
    scale_x_discrete(position = "top",
                     labels = c("0" = "Pred 0", "1" = "Pred 1")) +
    scale_y_discrete(labels = c("0" = "True 0", "1" = "True 1")) +
    labs(title = title, x = NULL, y = NULL) +
    theme_void(base_size = 11) +
    theme(
      plot.title = element_text(face = "bold", hjust = 0.5, size = 12,
                                margin = ggplot2::margin(b = 10)),
      axis.text = element_text(size = 9, color = "black"),
      axis.text.x.top = element_text(margin = ggplot2::margin(b = 5)),
      axis.text.y = element_text(margin = ggplot2::margin(r = 5)),

      plot.margin = ggplot2::margin(5, 10, 5, 10)
    )
  if (!show_y) p <- p + theme(axis.text.y = element_blank())
  p
}

chur_cm_tree <- plot_cm_bw(Chur_t1, "Decision Tree", TRUE)
chur_cm_nb <- plot_cm_bw(Chur_t2, "Naive Bayes", FALSE)
chur_cm_bag <- plot_cm_bw(Chur_t3, "Bagging", FALSE)
chur_cm_boost <- plot_cm_bw(Chur_t4, "Boosting", FALSE)
chur_cm_rf <- plot_cm_bw(Chur_t5, "Random Forest", FALSE)

grid.arrange(
  chur_cm_tree,
  chur_cm_nb,
  chur_cm_bag,
  chur_cm_boost,
  chur_cm_rf,
  nrow = 1,
  top = textGrob(
    "Q5 - CChurches: Confusion Matrices",
    gp = gpar(fontface = "bold", fontsize = 14)
  )
)

```

```

tel_cm_tree <- plot_cm_bw(Tel_t1, "Decision Tree", TRUE)
tel_cm_nb <- plot_cm_bw(Tel_t2, "Naive Bayes", FALSE)
tel_cm_bag <- plot_cm_bw(Tel_t3, "Bagging", FALSE)
tel_cm_boost <- plot_cm_bw(Tel_t4, "Boosting", FALSE)
tel_cm_rf <- plot_cm_bw(Tel_t5, "Random Forest", FALSE)

```

```

grid.arrange(
  tel_cm_tree,
  tel_cm_nb,
  tel_cm_bag,
  tel_cm_boost,
  tel_cm_rf,
  nrow = 1,
  top = textGrob(
    "Q5 - CTelevision: Confusion Matrices",
    gp = gpar(fontface = "bold", fontsize = 14)
  )
)

```

```

par_cm_tree <- plot_cm_bw(Par_t1, "Decision Tree", TRUE)
par_cm_nb <- plot_cm_bw(Par_t2, "Naive Bayes", FALSE)
par_cm_bag <- plot_cm_bw(Par_t3, "Bagging", FALSE)
par_cm_boost <- plot_cm_bw(Par_t4, "Boosting", FALSE)
par_cm_rf <- plot_cm_bw(Par_t5, "Random Forest", FALSE)

```

```

grid.arrange(
  par_cm_tree,
  par_cm_nb,
  par_cm_bag,
  par_cm_boost,
  par_cm_rf,
  nrow = 1,
  top = textGrob(
    "Q5 - CParliament: Confusion Matrices",
    gp = gpar(fontface = "bold", fontsize = 14)
  )
)

```

```

#----- Question 6 ROC + AUC
##### Q6 – ROC Curves & AUC: CChurches #####
# Decision Tree

```

```

par(
  mfrow = c(1,3),

  mar = c(4, 4, 3, 1),

  oma = c(0, 0, 2, 0),

  mgp = c(2.2, 0.7, 0),

  tcl = -0.3
)

```

```

chur_predtree_conf <- predict(
  churches_models$tree,

```

```

newdata = WD.test,

type = "vector"
)

chur_dt_pred <- ROCR::prediction(
  chur_predtree_conf[,2],
  WD.test$CChurches
)

chur_dt_perf <- ROCR::performance(
  chur_dt_pred,
  "tpr",
  "fpr"
)

# Naive Bayes

chur_prednb_conf <- e1071::predict(
  churches_models$nb,
  WD.test,
  type = "raw"
)

chur_nb_pred <- ROCR::prediction(
  chur_prednb_conf[,2],
  WD.test$CChurches
)

chur_nb_perf <- ROCR::performance(
  chur_nb_pred,
  "tpr",
  "fpr"
)

# Bagging

chur_predbag_conf <- predict.bagging(
  churches_models$bag,
  WD.test
)

chur_bag_pred <- ROCR::prediction(
  chur_predbag_conf$prob[,2],
  WD.test$CChurches
)

chur_bag_perf <- ROCR::performance(
  chur_bag_pred,
  "tpr",
  "fpr"
)

```

```

# Boosting

chur_predboost_conf <- predict.boosting(
  churches_models$boost,
  WD.test
)

chur_boost_pred <- ROCR::prediction(
  chur_predboost_conf$prob[,2],
  WD.test$CChurches
)

chur_boost_perf <- ROCR::performance(
  chur_boost_pred,
  "tpr",
  "fpr"
)

# Random Forest

chur_predrf_conf <- predict(
  churches_models$rf,
  WD.test,
  type = "prob"
)

chur_rf_pred <- ROCR::prediction(
  chur_predrf_conf[,2],
  WD.test$CChurches
)

chur_rf_perf <- ROCR::performance(
  chur_rf_pred,
  "tpr",
  "fpr"
)

# AUC Values

roc_dt_chur <- pROC::roc(
  WD.test$CChurches,
  chur_predtree_conf[,2]
)

roc_nb_chur <- pROC::roc(
  WD.test$CChurches,
  chur_prednb_conf[,2]
)

roc_bag_chur <- pROC::roc(
  WD.test$CChurches,
  chur_predbag_conf$prob[,2]
)

roc_boost_chur <- pROC::roc(

```

```

    WD.test$CChurches,
    chur_predboost_conf$prob[,2]
)

roc_rf_chur <- pROC::roc(
  WD.test$CChurches,
  chur_predrf_conf[,2]
)

# ROC Plot

plot(
  chur_dt_perf,

  col = "#E74C3C",
  lwd = 3,

  main = "ROC - CChurches Classifiers",
  sub = paste0("(test n = ", nrow(WD.test), ")"),

  xlab = "False Positive Rate",
  ylab = "True Positive Rate",

  cex.main = 1.6,
  cex.sub = 1.1,
  cex.lab = 1.2,
  cex.axis = 1.1,

  las = 1
)

grid(
  col = "grey85",
  lty = 1
)

# Naive Bayes
plot(
  chur_nb_perf,
  add = TRUE,
  col = "#9B59B6",
  lwd = 3
)

# Bagging
plot(
  chur_bag_perf,
  add = TRUE,
  col = "#27AE60",
  lwd = 3
)

# Boosting
plot(
  chur_boost_perf,
  add = TRUE,
  col = "#F39C12",

```

```

    lwd = 3
)

# Random Forest
plot(
  chur_rf_perf,
  add = TRUE,
  col = "#2980B9",
  lwd = 3
)

# Random Guessing Line

abline(
  a = 0,
  b = 1,

  col = "grey50",
  lty = 2,
  lwd = 2
)

# Legend

legend(
  "bottomright",

  legend = c(
    paste0(
      "Decision Tree (AUC = ",
      round(pROC::auc(roc_dt_chur), 3),
      ") "
    ),
    paste0(
      "Naive Bayes (AUC = ",
      round(pROC::auc(roc_nb_chur), 3),
      ") "
    ),
    paste0(
      "Bagging (AUC = ",
      round(pROC::auc(roc_bag_chur), 3),
      ") "
    ),
    paste0(
      "Boosting (AUC = ",
      round(pROC::auc(roc_boost_chur), 3),
      ") "
    ),
    paste0(
      "Random Forest (AUC = ",
      round(pROC::auc(roc_rf_chur), 3),
      ") "
    )
  ),

```

```

      "Chance (AUC = 0.500)"
    ),

    col = c(
      "#E74C3C",
      "#9B59B6",
      "#27AE60",
      "#F39C12",
      "#2980B9",
      "grey50"
    ),

    lwd = c(
      3, 3, 3, 3, 3,
      2
    ),

    lty = c(
      1, 1, 1, 1, 1,
      2
    ),

    cex = 0.95,

    bty = "n"
  )

##### Q6 – ROC Curves & AUC: CTelevision #####

# Decision Tree

tel_predtree_conf <- predict(
  television_models$tree,
  WD.test,
  type = "vector"
)

tel_dt_pred <- ROCR::prediction(
  tel_predtree_conf[,2],
  WD.test$CTelevision
)

tel_dt_perf <- ROCR::performance(
  tel_dt_pred,
  "tpr",
  "fpr"
)

# Naive Bayes

tel_prednb_conf <- predict(
  television_models$nb,
  WD.test,
  type = "raw"
)

tel_nb_pred <- ROCR::prediction(
  tel_prednb_conf[,2],
  WD.test$CTelevision

```



```

)

tel_nb_perf <- ROCR::performance(
  tel_nb_pred,
  "tpr",
  "fpr"
)

# Bagging

tel_predbag_conf <- predict.bagging(
  television_models$bag,
  WD.test
)

tel_bag_pred <- ROCR::prediction(
  tel_predbag_conf$prob[,2],
  WD.test$CTelevision
)

tel_bag_perf <- ROCR::performance(
  tel_bag_pred,
  "tpr",
  "fpr"
)

# Boosting

tel_predboost_conf <- predict.boosting(
  television_models$boost,
  WD.test
)

tel_boost_pred <- ROCR::prediction(
  tel_predboost_conf$prob[,2],
  WD.test$CTelevision
)

tel_boost_perf <- ROCR::performance(
  tel_boost_pred,
  "tpr",
  "fpr"
)

# Random Forest

tel_predrf_conf <- predict(
  television_models$rf,
  WD.test,
  type = "prob"
)

tel_rf_pred <- ROCR::prediction(
  tel_predrf_conf[,2],
  WD.test$CTelevision
)

```

```

tel_rf_perf <- ROCR::performance(
  tel_rf_pred,
  "tpr",
  "fpr"
)

# AUC Values

roc_dt_tel <- pROC::roc(
  WD.test$CTelevision,
  tel_predtree_conf[,2]
)

roc_nb_tel <- pROC::roc(
  WD.test$CTelevision,
  tel_prednb_conf[,2]
)

roc_bag_tel <- pROC::roc(
  WD.test$CTelevision,
  tel_predbag_conf$prob[,2]
)

roc_boost_tel <- pROC::roc(
  WD.test$CTelevision,
  tel_predboost_conf$prob[,2]
)

roc_rf_tel <- pROC::roc(
  WD.test$CTelevision,
  tel_predrf_conf[,2]
)

# ROC Plot

plot(
  tel_dt_perf,

  col = "#E74C3C",
  lwd = 3,

  main = "ROC - CTelevision Classifiers",
  sub = paste0("(test n = ", nrow(WD.test), ")"),

  xlab = "False Positive Rate",
  ylab = "True Positive Rate",

  cex.main = 1.6,
  cex.sub = 1.1,
  cex.lab = 1.2,
  cex.axis = 1.1,

  las = 1
)

grid(

```

```

    col = "grey85",
    lty = 1
)

# Naive Bayes
plot(
  tel_nb_perf,
  add = TRUE,
  col = "#9B59B6",
  lwd = 3
)

# Bagging
plot(
  tel_bag_perf,
  add = TRUE,
  col = "#27AE60",
  lwd = 3
)

# Boosting
plot(
  tel_boost_perf,
  add = TRUE,
  col = "#F39C12",
  lwd = 3
)

# Random Forest
plot(
  tel_rf_perf,
  add = TRUE,
  col = "#2980B9",
  lwd = 3
)

# Random Guessing Line

abline(
  a = 0,
  b = 1,

  col = "grey50",
  lty = 2,
  lwd = 2
)

# Legend

legend(
  "bottomright",

  legend = c(
    paste0(
      "Decision Tree (AUC = ",
      round(pROC::auc(roc_dt_tel), 3),
      ")"
    )
  )

```

```

),

paste0(
  "Naive Bayes (AUC = ",
  round(pROC::auc(roc_nb_tel), 3),
  ") "
),

paste0(
  "Bagging (AUC = ",
  round(pROC::auc(roc_bag_tel), 3),
  ") "
),

paste0(
  "Boosting (AUC = ",
  round(pROC::auc(roc_boost_tel), 3),
  ") "
),

paste0(
  "Random Forest (AUC = ",
  round(pROC::auc(roc_rf_tel), 3),
  ") "
),

  "Random Guessing (AUC = 0.500) "
),

col = c(
  "#E74C3C",
  "#9B59B6",
  "#27AE60",
  "#F39C12",
  "#2980B9",
  "grey50"
),

lwd = c(
  3, 3, 3, 3, 3,
  2
),

lty = c(
  1, 1, 1, 1, 1,
  2
),

cex = 0.95,

bty = "n"
)

##### Q6 – ROC Curves & AUC: CParliament #####

# Decision Tree

par_predtree_conf <- predict(
  parliament_models$tree,
  WD.test,

```

```

    type = "vector"
  )

par_dt_pred <- ROCR::prediction(
  par_predtree_conf[,2],
  WD.test$CParliament
)

par_dt_perf <- ROCR::performance(
  par_dt_pred,
  "tpr",
  "fpr"
)

# Naive Bayes

par_prednb_conf <- predict(
  parliament_models$nb,
  WD.test,
  type = "raw"
)

par_nb_pred <- ROCR::prediction(
  par_prednb_conf[,2],
  WD.test$CParliament
)

par_nb_perf <- ROCR::performance(
  par_nb_pred,
  "tpr",
  "fpr"
)

# Bagging

par_predbag_conf <- predict.bagging(
  parliament_models$bag,
  WD.test
)

par_bag_pred <- ROCR::prediction(
  par_predbag_conf$prob[,2],
  WD.test$CParliament
)

par_bag_perf <- ROCR::performance(
  par_bag_pred,
  "tpr",
  "fpr"
)

# Boosting

par_predboost_conf <- predict.boosting(
  parliament_models$boost,
  WD.test
)

```

```

par_boost_pred <- ROCR::prediction(
  par_predboost_conf$prob[,2],
  WD.test$CParliament
)

par_boost_perf <- ROCR::performance(
  par_boost_pred,
  "tpr",
  "fpr"
)

# Random Forest

par_predrf_conf <- predict(
  parliament_models$rf,
  WD.test,
  type = "prob"
)

par_rf_pred <- ROCR::prediction(
  par_predrf_conf[,2],
  WD.test$CParliament
)

par_rf_perf <- ROCR::performance(
  par_rf_pred,
  "tpr",
  "fpr"
)

# AUC Values

roc_dt_par <- pROC::roc(
  WD.test$CParliament,
  par_predtree_conf[,2]
)

roc_nb_par <- pROC::roc(
  WD.test$CParliament,
  par_prednb_conf[,2]
)

roc_bag_par <- pROC::roc(
  WD.test$CParliament,
  par_predbag_conf$prob[,2]
)

roc_boost_par <- pROC::roc(
  WD.test$CParliament,
  par_predboost_conf$prob[,2]
)

roc_rf_par <- pROC::roc(
  WD.test$CParliament,
  par_predrf_conf[,2]
)

```

```

# ROC Plot

plot(
  par_dt_perf,

  col = "#E74C3C",
  lwd = 3,

  main = "ROC - CParliament Classifiers",
  sub = paste0("(test n = ", nrow(WD.test), ")"),

  xlab = "False Positive Rate",
  ylab = "True Positive Rate",

  cex.main = 1.6,
  cex.sub = 1.1,
  cex.lab = 1.2,
  cex.axis = 1.1,

  las = 1
)

grid(
  col = "grey85",
  lty = 1
)

# Naive Bayes
plot(
  par_nb_perf,
  add = TRUE,
  col = "#9B59B6",
  lwd = 3
)

# Bagging
plot(
  par_bag_perf,
  add = TRUE,
  col = "#27AE60",
  lwd = 3
)

# Boosting
plot(
  par_boost_perf,
  add = TRUE,
  col = "#F39C12",
  lwd = 3
)

# Random Forest
plot(
  par_rf_perf,
  add = TRUE,
  col = "#2980B9",
  lwd = 3
)

```

```

# Random Guessing Line
abline(
  a = 0,
  b = 1,

  col = "grey50",
  lty = 2,
  lwd = 2
)

# Legend
legend(
  "bottomright",

  legend = c(
    paste0(
      "Decision Tree (AUC = ",
      round(pROC::auc(roc_dt_par), 3),
      ") "
    ),
    paste0(
      "Naive Bayes (AUC = ",
      round(pROC::auc(roc_nb_par), 3),
      ") "
    ),
    paste0(
      "Bagging (AUC = ",
      round(pROC::auc(roc_bag_par), 3),
      ") "
    ),
    paste0(
      "Boosting (AUC = ",
      round(pROC::auc(roc_boost_par), 3),
      ") "
    ),
    paste0(
      "Random Forest (AUC = ",
      round(pROC::auc(roc_rf_par), 3),
      ") "
    ),
    "Random Guessing (AUC = 0.500) "
  ),

  col = c(
    "#E74C3C",
    "#9B59B6",
    "#27AE60",
    "#F39C12",
    "#2980B9",
    "grey50"
  ),

  lwd = c(
    3, 3, 3, 3, 3,
    2
  ),

```



```

lty = c(
  1, 1, 1, 1, 1,
  2
),

cex = 0.95,

bty = "n"
)

mtext(
  "Q6 - ROC Curves & AUC Comparison",
  outer = TRUE,
  cex = 1.6,
  font = 2
)

#q6 report auc table

#cchurch auc comparison
Chur_auc <- data.frame(
  Classifier = c("Decision Tree",
                 "Naive Bayes",
                 "Bagging",
                 "Boosting",
                 "Random Forest"),

  AUC = c(
    auc(roc_dt_chur),
    auc(roc_nb_chur),
    auc(roc_bag_chur),
    auc(roc_boost_chur),
    auc(roc_rf_chur)
  )
)

best_auc_row_chur <- which.max(Chur_auc$AUC)
kable(
  Chur_auc,
  digits = 3,

  caption = "
<span style='font-size:18px;
              font-weight:bold;
              color:#2F5597;'>
CChurches Classifiers: AUC Comparison
</span>
",

  escape = FALSE
) %>%

kable_styling(
  full_width = TRUE,

  html_font = "Arial",

  bootstrap_options = c(

```

```

      "hover",
      "condensed"
    ),

    position = "left"
  ) %>%

  row_spec(
    0,
    background = "#D9E2F3",
    bold = TRUE
  ) %>%

  row_spec(
    best_auc_row_chur,
    background = "#FFF2CC",
    bold = TRUE
  )

#ctelelevision auc comparison
Tel_auc <- data.frame(
  Classifier = c("Decision Tree",
                 "Naive Bayes",
                 "Bagging",
                 "Boosting",
                 "Random Forest"),

  AUC = c(
    auc(roc_dt_tel),
    auc(roc_nb_tel),
    auc(roc_bag_tel),
    auc(roc_boost_tel),
    auc(roc_rf_tel)
  )
)

best_auc_row_tel <- which.max(Tel_auc$AUC)

kable(
  Tel_auc,
  digits = 3,
  caption = "<span style='font-size:18px; font-weight:bold; color:#2F5597;'>
CTelevision Classifiers: AUC Comparison
</span>"
,
  escape = FALSE
) %>%
  kable_styling(
    full_width = TRUE,
    bootstrap_options = c(
      "hover",
      "condensed"
    ),
    position = "center"
  ) %>%
  row_spec(
    0,
    background = "#D9E2F3",
    bold = TRUE
  ) %>%
  row_spec(

```

```

    best_auc_row_tel,
    background = "#FFF2CC",
    bold = TRUE
  )

#cparliament auc comparison
Par_auc <- data.frame(
  Classifier = c("Decision Tree",
                 "Naive Bayes",
                 "Bagging",
                 "Boosting",
                 "Random Forest"),

  AUC = c(
    auc(roc_dt_par),
    auc(roc_nb_par),
    auc(roc_bag_par),
    auc(roc_boost_par),
    auc(roc_rf_par)
  )
)

best_auc_row_par <- which.max(Par_auc$AUC)

kable(
  Par_auc,
  digits = 3,
  caption = "<span style='font-size:18px; font-weight:bold; color:#2F5597;'>
CParliament Classifiers: AUC Comparison
</span>"
,
  escape = FALSE
) %>%
kable_styling(
  full_width = TRUE,
  bootstrap_options = c(
    "hover",
    "condensed"
  ),
  position = "center"
) %>%
row_spec(
  0,
  background = "#D9E2F3",
  bold = TRUE
) %>%
row_spec(
  best_auc_row_par,
  background = "#FFF2CC",
  bold = TRUE
)

AUC_table <- data.frame(
  Classifier = c("Decision Tree",
                 "Naive Bayes",
                 "Bagging",
                 "Boosting",
                 "Random Forest"),

```

```

CChurches = c(
  auc(roc_dt_chur),
  auc(roc_nb_chur),
  auc(roc_bag_chur),
  auc(roc_boost_chur),
  auc(roc_rf_chur)
),

CTelevision = c(
  auc(roc_dt_tel),
  auc(roc_nb_tel),
  auc(roc_bag_tel),
  auc(roc_boost_tel),
  auc(roc_rf_tel)
),

CParliament = c(
  auc(roc_dt_par),
  auc(roc_nb_par),
  auc(roc_bag_par),
  auc(roc_boost_par),
  auc(roc_rf_par)
)
)

kable(AUC_table, digits = 3,
      caption = "AUC Comparison Across Classifiers and Class Variables") %>%
  kable_styling(full_width = TRUE) %>%
  row_spec(0, background = "#D9E2F3")

```

Q7- Cross-Classifier Comparison

Build F1 Matrix

```

F1_matrix <- rbind(
  CChurches = Chur_metrics[, "F1"],
  CTelevision = Tel_metrics[, "F1"],
  CParliament = Par_metrics[, "F1"]
)

```

```

short_names <- c(
  "DT",
  "NB",
  "Bag",
  "Boost",
  "RF"
)

```

```
colnames(F1_matrix) <- short_names
```

Build AUC Matrix

```

AUC_matrix <- rbind(
  CChurches = Chur_auc$AUC,
  CTelevision = Tel_auc$AUC,
  CParliament = Par_auc$AUC
)

```

```

)

colnames(AUC_matrix) <- short_names

# Layout

par(
  mfrow = c(1,2),

  mar = c(6,7,4,3),

  oma = c(0,0,3,0),

  mgp = c(2,0.7,0)
)

# F1 Heatmap

image.plot(
  1:ncol(F1_matrix),
  1:nrow(F1_matrix),

  t(F1_matrix[nrow(F1_matrix):1, ]),

  col = colorRampPalette(c(
    "#F7FBFF",
    "#6BAED6",
    "#08306B"
  ))(100),

  zlim = c(0,1),

  axes = FALSE,

  main = "F1-score",

  xlab = "",
  ylab = "",

  xaxs = "i",
  yaxs = "i",

  legend.width = 1.2,

  legend.shrink = 0.7
)

axis(
  1,
  at = 1:ncol(F1_matrix),
  labels = colnames(F1_matrix),
  las = 1,
  cex.axis = 1.1
)

axis(

```

```

2,
at = 1:nrow(F1_matrix),
labels = rev(rownames(F1_matrix)),
las = 1,
cex.axis = 1.1
)

# Add Numbers
for(i in 1:nrow(F1_matrix)) {

  for(j in 1:ncol(F1_matrix)) {

    value <- F1_matrix[i,j]

    text(
      j,
      nrow(F1_matrix)-i+1,

      labels = sprintf("%.3f", value),

      col = ifelse(value > 0.65, "white", "black"),

      cex = 1.1,
      font = 2
    )
  }
}

```

```

# AUC Heatmap
image.plot(
  1:ncol(AUC_matrix),
  1:nrow(AUC_matrix),

  t(AUC_matrix[nrow(AUC_matrix):1, ]),

  col = colorRampPalette(c(
    "#F7FCF5",
    "#74C476",
    "#00441B"
  ))(100),

  zlim = c(0,1),

  axes = FALSE,

  main = "AUC",

  xaxs = "i",
  yaxs = "i",

  xlab = "",
  ylab = "",

  legend.width = 1.2,

  legend.shrink = 0.7
)

```

```
axis(
```

```

1,
at = 1:ncol(AUC_matrix),
labels = colnames(AUC_matrix),
las = 1,
cex.axis = 1.1
)

axis(
  2,
  at = 1:nrow(AUC_matrix),
  labels = rev(rownames(AUC_matrix)),
  las = 1,
  cex.axis = 1.1
)

# Add Numbers
for(i in 1:nrow(AUC_matrix)) {

  for(j in 1:ncol(AUC_matrix)) {

    value <- AUC_matrix[i,j]

    text(
      j,
      nrow(AUC_matrix)-i+1,

      labels = sprintf("%.3f", value),

      col = ifelse(value > 0.72, "white", "black"),

      cex = 1.1,
      font = 2
    )
  }
}

mtext(
  "DT = Decision Tree    |    NB = Naive Bayes    |    Bag = Bagging    |    Boost =
Boosting    |    RF = Random Forest",

  side = 1,

  outer = TRUE,

  line = -1,

  cex = 0.9
)

# Q7Title

mtext(
  "Q7: Cross-Classfier Comparison (F1 & AUC)",
  outer = TRUE,
  cex = 1.6,
  font = 2
)

par(mfrow = c(1,1))

```

Q8 – Variable Importance Tables

```
create_importance_table <- function(bag_imp,
                                    boost_imp,
                                    rf_imp,
                                    title) {

  rf_values <- rf_imp[,1]

  bag_df <- data.frame(
    Variable = names(bag_imp),
    Bagging = as.numeric(bag_imp)
  )

  boost_df <- data.frame(
    Variable = names(boost_imp),
    Boosting = as.numeric(boost_imp)
  )

  rf_df <- data.frame(
    Variable = rownames(rf_imp),
    RandomForest = as.numeric(rf_values)
  )

  # Merge tables

  imp_table <- merge(
    bag_df,
    boost_df,
    by = "Variable"
  )

  imp_table <- merge(
    imp_table,
    rf_df,
    by = "Variable"
  )

  # Create rankings

  imp_table$BagRank <- rank(
    -imp_table$Bagging
  )

  imp_table$BoostRank <- rank(
    -imp_table$Boosting
  )

  imp_table$RFRank <- rank(
    -imp_table$RandomForest
  )

  # Average ranking

  imp_table$AvgRank <- rowMeans(
    imp_table[, c(
      "BagRank",
      "BoostRank",
      "RFRank"
    )]
```



```

    )]
  )

# Sort by average ranking

imp_table <- imp_table %>%
  arrange(AvgRank)

# Round values

imp_table$Bagging <- round(
  imp_table$Bagging,
  2
)

imp_table$Boosting <- round(
  imp_table$Boosting,
  2
)

imp_table$RandomForest <- round(
  imp_table$RandomForest,
  2
)

imp_table$BagRank <- round(
  imp_table$BagRank,
  1
)

imp_table$BoostRank <- round(
  imp_table$BoostRank,
  1
)

imp_table$RFRank <- round(
  imp_table$RFRank,
  1
)

imp_table$AvgRank <- round(
  imp_table$AvgRank,
  2
)

rownames(imp_table) <- NULL

# Create styled table

kable(
  imp_table,

  align = "c",

  escape = FALSE,

  caption = paste0(

    "<span style='font-size:28px;
    font-weight:bold;

```

```

        color:black;'>",
        title,
        "</span><br>",

        "<span style='font-size:15px;
        color:#666666;'>
        Variables ranked by average ranking across ensemble models;
        top 5 highlighted in yellow
        </span>"
    )
) %>%

kable_styling(

    bootstrap_options = c(
        "striped",
        "condensed"
    ),

    full_width = T,

    position = "center",

    font_size = 15
) %>%

row_spec(
    0,
    bold = TRUE,
    color = "white",
    background = "#2F75B5"
) %>%

row_spec(
    1:5,
    background = "#FFF2CC",
    bold = TRUE
)
}

# CChurches

var_imp_churches<-create_importance_table(

    churches_models$bag$importance,

    churches_models$boost$importance,

    importance(churches_models$rf),

    "Q8: Variable Importance: CChurches"
)

save_kable(
    var_imp_churches,
    "CChurches_importance.png",
    density = 300
)

```

```

# CTelevision

var_imp_television <- create_importance_table(
  television_models$bag$importance,
  television_models$boost$importance,
  importance(television_models$rfr),
  "Q8: Variable Importance: CTelevision"
)

save_kable(
  var_imp_television,
  "CTelevision_importance.png",
  density = 300
)

# CParliament

var_imp_parliament <- create_importance_table(
  parliament_models$bag$importance,
  parliament_models$boost$importance,
  importance(parliament_models$rfr),
  "Q8: Variable Importance: CParliament"
)

save_kable(
  var_imp_parliament,
  "CParliament_importance.png",
  density = 300
)

#Q9
prop.table(table(WD.train$CChurches))
prop.table(table(WD.test$CChurches))

prop.table(table(WD.train$CTelevision))
prop.table(table(WD.test$CTelevision))

prop.table(table(WD.train$CParliament))
prop.table(table(WD.test$CParliament))

#Q10 find out the worst ensemble model using avg F1-score

# Build Average F1 Table

ensemble_f1 <- data.frame(

```

```

Classifier = c(
  "Bagging",
  "Boosting",
  "Random Forest"
),

CChurches = c(
  Chur_metrics["Bagging", "F1"],
  Chur_metrics["Boosting", "F1"],
  Chur_metrics["RandomForest", "F1"]
),

CTelevision = c(
  Tel_metrics["Bagging", "F1"],
  Tel_metrics["Boosting", "F1"],
  Tel_metrics["RandomForest", "F1"]
),

CParliament = c(
  Par_metrics["Bagging", "F1"],
  Par_metrics["Boosting", "F1"],
  Par_metrics["RandomForest", "F1"]
)
)

# Calculate Average F1

ensemble_f1$Average_F1 <- rowMeans(
  ensemble_f1[,2:4]
)

ensemble_f1$Average_F1 <- round(
  ensemble_f1$Average_F1,
  3
)

# Highlight Worst

worst_ensemble <- which.min(
  ensemble_f1$Average_F1
)

# Styled Table
kable(
  ensemble_f1,

  digits = 3,

  caption = "
<span style='font-size:22px;
                font-weight:bold;
                color:#2F5597;'>
Q10: Average F1-score Across Ensemble Classifiers
</span>
",

  escape = FALSE
) %>%

```

```

kable_styling(
  full_width = TRUE,

  bootstrap_options = c(
    'striped',
    'condensed'
  ),

  position = "center"
) %>%

row_spec(
  0,
  background = "#D9E2F3",
  bold = TRUE
) %>%

row_spec(
  worst_ensemble,
  background = "#FFF2CC",
  bold = TRUE
)

avg_auc <- data.frame(

  Classifier = Chur_auc$Classifier,

  Avg_AUC = (
    Chur_auc$AUC +
    Tel_auc$AUC +
    Par_auc$AUC
  ) / 3
)

avg_auc_ensemble <- subset(
  avg_auc,

  Classifier %in% c(
    "Bagging",
    "Boosting",
    "Random Forest"
  )
)

avg_ensemble_metrics <- data.frame(

  Classifier = c(
    "Bagging",
    "Boosting",
    "Random Forest"
  ),

  CChurches = c(
    Chur_metrics["Bagging", "F1"],
    Chur_metrics["Boosting", "F1"],
    Chur_metrics["RandomForest", "F1"]
  ),

  CTelevision = c(

```

```

    Tel_metrics["Bagging", "F1"],
    Tel_metrics["Boosting", "F1"],
    Tel_metrics["RandomForest", "F1"]
  ),

  CParliament = c(
    Par_metrics["Bagging", "F1"],
    Par_metrics["Boosting", "F1"],
    Par_metrics["RandomForest", "F1"]
  )
)

avg_ensemble_metrics$Avg_F1 <- rowMeans(
  avg_ensemble_metrics[,2:4]
)

##### Average AUC Table #####

avg_auc_table <- data.frame(

  Classifier = c(
    "Bagging",
    "Boosting",
    "Random Forest"
  ),

  CChurches = c(
    pROC::auc(roc_bag_chur),
    pROC::auc(roc_boost_chur),
    pROC::auc(roc_rf_chur)
  ),

  CTelevision = c(
    pROC::auc(roc_bag_tel),
    pROC::auc(roc_boost_tel),
    pROC::auc(roc_rf_tel)
  ),

  CParliament = c(
    pROC::auc(roc_bag_par),
    pROC::auc(roc_boost_par),
    pROC::auc(roc_rf_par)
  )
)

# Average AUC
avg_auc_table$Avg_AUC <- rowMeans(
  avg_auc_table[,2:4]
)

# Round
avg_auc_table[,2:5] <- round(
  avg_auc_table[,2:5],
  3
)

kable(

  avg_auc_table,

  digits = 3,

```

```

caption = "
<span style='font-size:20px;
          font-weight:bold;
          color:#2F5597;'>
Q10: Average AUC Across Ensemble Classifiers
</span>
",

escape = FALSE

) %>%

kable_styling(
  full_width = TRUE,
  bootstrap_options = c("hover", "condensed"),
  position = "center"
) %>%

row_spec(
  0,
  background = "#D9E2F3",
  bold = TRUE
) %>%

row_spec(
  worst_ensemble,
  background = "#FFF2CC",
  bold = TRUE
)

#attribute selection
create_importance_df <- function(bag_imp,
                                boost_imp,
                                rf_imp) {

  rf_values <- rf_imp[,1]

  bag_df <- data.frame(
    Variable = names(bag_imp),
    Bagging = as.numeric(bag_imp)
  )

  boost_df <- data.frame(
    Variable = names(boost_imp),
    Boosting = as.numeric(boost_imp)
  )

  rf_df <- data.frame(
    Variable = rownames(rf_imp),
    RandomForest = as.numeric(rf_values)
  )

  imp_table <- merge(
    bag_df,
    boost_df,
    by = "Variable"
  )

  imp_table <- merge(

```

```

    imp_table,
    rf_df,
    by = "Variable"
  )

  imp_table$BagRank <- rank(-imp_table$Bagging)

  imp_table$BoostRank <- rank(-imp_table$Boosting)

  imp_table$RFRank <- rank(-imp_table$RandomForest)

  imp_table$AvgRank <- rowMeans(
    imp_table[, c(
      "BagRank",
      "BoostRank",
      "RFRank"
    )]
  )
}

imp_table <- imp_table %>%
  arrange(AvgRank)

return(imp_table)
}
all_importance <- list()

all_importance$CChurches <- create_importance_df(
  churches_models$bag$importance,
  churches_models$boost$importance,
  importance(churches_models$rf)
)

all_importance$CTelevision <- create_importance_df(
  television_models$bag$importance,
  television_models$boost$importance,
  importance(television_models$rf)
)

all_importance$CParliament <- create_importance_df(
  parliament_models$bag$importance,
  parliament_models$boost$importance,
  importance(parliament_models$rf)
)

confidence_vars

pred_cols <- setdiff(
  names(WD.train),

  c(
    "Country",
    "Wave",
    confidence_vars
  )
)

imp_col <- switch(
  worst_ensemble,

```



```

"Bagging" = "Bagging",

"Boosting" = "Boosting",

"Random Forest" = "RandomForest"
)

avg_imp <- sapply(pred_cols, function(v) {
  mean(
    sapply(confidence_vars, function(t)
      all_importance[[t]][
        all_importance[[t]]$Variable == v,
        imp_col
      ]
    )
  )
})

avg_imp <- sort(avg_imp, decreasing = TRUE)
avg_imp

worst_ensemble <- ensemble_f1$Classifier[
  which.min(ensemble_f1$Average_F1)
]

top_features <- names(avg_imp)[1:min(20, length(avg_imp))]
top_features

worst_ensemble <- ensemble_f1$Classifier[
  which.min(ensemble_f1$Average_F1)
]

top_features <- names(avg_imp)[1:min(20, length(avg_imp))]
top_features

targets <- confidence_vars

# Q10: Improve Bagging (Worst Ensemble)

# top 20 features
worst_metric_key <- "Bagging"
imp_col <- worst_metric_key

avg_imp <- sapply(pred_cols, function(v) {
  mean(sapply(confidence_vars, function(t)
    all_importance[[t]][all_importance[[t]]$Variable == v, imp_col]
  ))
})
avg_imp <- sort(avg_imp, decreasing = TRUE)
top_features <- names(avg_imp)[1:min(20, length(avg_imp))]
cat("Top features selected:\n"); print(top_features)

# Q10: Improve Bagging (Worst Ensemble)

```

```

stopifnot(exists("get_metrics"))
stopifnot(exists("top_features"))
stopifnot(exists("WD.train"))
stopifnot(exists("WD.test"))
stopifnot(exists("confidence_vars"))

# cv
set.seed(33235422)
k_folds <- 5
fold_ids <- sample(rep(1:k_folds, length.out = nrow(WD.train)))
param_grid <- c(50, 100, 150, 200, 300)

# container
improved_models <- list()
improved_metrics <- list()
improved_probs <- list()

# 3 targets
for (tgt in confidence_vars) {
  cat("\nProcessing", tgt, "... \n")
  keep_cols <- c(top_features, tgt)

  # pick best mfinal for 5-fold CV
  cv_f1 <- sapply(param_grid, function(param) {
    fold_f1 <- sapply(1:k_folds, function(f) {
      tr_fold <- WD.train[fold_ids != f, keep_cols]
      va_fold <- WD.train[fold_ids == f, keep_cols]
      fml <- as.formula(paste(tgt, "~ ."))

      mod <- adabag::bagging(fml, data = tr_fold, mfinal = param)
      pred_class <- predict(mod, newdata = va_fold)$class

      cm_fold <- table(Predicted_Class = pred_class,
                       Actual_Class = va_fold[[tgt]])
      get_metrics(cm_fold) ["F1"]
    })
    mean(fold_f1)
  })

  best_param <- param_grid[which.max(cv_f1)]
  cat(" Best mfinal:", best_param, " (CV F1:", round(max(cv_f1), 4), ") \n")

  # use best mfinal on train
  tr_final <- WD.train[, keep_cols]
  te_final <- WD.test[, keep_cols]
  fml <- as.formula(paste(tgt, "~ ."))

  improved_models[[tgt]] <- adabag::bagging(fml, data = tr_final, mfinal =
best_param)
  pred_out <- predict(improved_models[[tgt]], newdata = te_final)

  cm_improved <- table(Predicted_Class = pred_out$class,
                       Actual_Class = WD.test[[tgt]])
  improved_metrics[[tgt]] <- get_metrics(cm_improved)
  improved_probs[[tgt]] <- pred_out$prob[, 2]
}

str(improved_metrics)

```

```

# Original/Improved probability + true label

orig_probs <- list(
  CChurches = chur_predbag_conf$prob[, 2],
  CTelevision = tel_predbag_conf$prob[, 2],
  CParliament = par_predbag_conf$prob[, 2]
)

true_labels <- list(
  CChurches = WD.test$CChurches,
  CTelevision = WD.test$CTelevision,
  CParliament = WD.test$CParliament
)

orig_metrics_map <- list(
  CChurches = Chur_metrics,
  CTelevision = Tel_metrics,
  CParliament = Par_metrics
)

# F1 comparison (Original vs Improved, 3 targets)

f1_compare <- do.call(rbind, lapply(targets, function(tgt) {
  f1_o <- as.numeric(orig_metrics_map[[tgt]]["Bagging", "F1"])
  f1_i <- as.numeric(improved_metrics[[tgt]]["F1"])
  data.frame(
    Target = tgt,
    F1_Original = round(f1_o, 4),
    F1_Improved = round(f1_i, 4),
    Delta_F1 = round(f1_i - f1_o, 4),
    Relative_Gain_pct = round((f1_i - f1_o) / max(f1_o, 1e-9) * 100, 2)
  )
}))
rownames(f1_compare) <- NULL

cat("\n===== F1 Comparison Table =====\n")
print(f1_compare)

best_f1_row <- which.max(f1_compare$Delta_F1)

f1_kbl <- kable(
  f1_compare, digits = 3,
  caption = "<span style='font-size:20px; font-weight:bold; color:#2F5597;'>
F1 Comparison: Original Bagging vs Improved Bagging
</span>",
  escape = FALSE
) %>%
  kable_styling(full_width = TRUE,
    bootstrap_options = c("hover", "condensed"),
    position = "center") %>%
  row_spec(0, background = "#D9E2F3", bold = TRUE) %>%
  row_spec(best_f1_row, background = "#FFF2CC", bold = TRUE)
print(f1_kbl)

# AUC comapriosn

auc_compare <- do.call(rbind, lapply(targets, function(tgt) {
  auc_o <- as.numeric(pROC::auc(pROC::roc(true_labels[[tgt]],
    orig_probs[[tgt]],
    quiet = TRUE)))

```

```

auc_i <- as.numeric(pROC::auc(pROC::roc(true_labels[[tgt]],
                                       improved_probs[[tgt]],
                                       quiet = TRUE)))

data.frame(
  Target      = tgt,
  AUC_Original = round(auc_o, 4),
  AUC_Improved = round(auc_i, 4),
  Delta_AUC    = round(auc_i - auc_o, 4)
)
)))
rownames(auc_compare) <- NULL

cat("\n===== AUC Comparison Table =====\n")
print(auc_compare)

best_auc_row <- which.max(auc_compare$Delta_AUC)

auc_kbl <- kable(
  auc_compare, digits = 3,
  caption = "<span style='font-size:20px; font-weight:bold; color:#2F5597;'>
AUC Comparison: Original Bagging vs Improved Bagging
</span>",
  escape = FALSE
) %>%
  kable_styling(full_width = TRUE,
                bootstrap_options = c("hover", "condensed"),
                position = "center") %>%
  row_spec(0, background = "#D9E2F3", bold = TRUE) %>%
  row_spec(best_auc_row, background = "#FFF2CC", bold = TRUE)
print(auc_kbl)

# ROC curves: 3 panels

par(mfrow = c(1, 3),
    mar = c(4.2, 4.2, 3.5, 1),
    oma = c(0, 0, 3, 0),
    mgp = c(2.3, 0.7, 0))

for (tgt in targets) {

  # Original
  pred_o <- ROCR::prediction(orig_probs[[tgt]], true_labels[[tgt]])
  perf_o <- ROCR::performance(pred_o, "tpr", "fpr")
  auc_o <- as.numeric(pROC::auc(pROC::roc(true_labels[[tgt]],
                                       orig_probs[[tgt]],
                                       quiet = TRUE)))

  # Improved
  pred_i <- ROCR::prediction(improved_probs[[tgt]], true_labels[[tgt]])
  perf_i <- ROCR::performance(pred_i, "tpr", "fpr")
  auc_i <- as.numeric(pROC::auc(pROC::roc(true_labels[[tgt]],
                                       improved_probs[[tgt]],
                                       quiet = TRUE)))

  # Original: dashed line
  plot(perf_o,
       col = "#E07A3F", lwd = 3, lty = 2,
       main = paste("ROC -", tgt),
       sub = paste0("(test n = ", length(true_labels[[tgt]]), ")"),
       xlab = "False Positive Rate",
       ylab = "True Positive Rate",

```

```

    cex.main = 1.4, cex.lab = 1.1, cex.axis = 1.0,
    las = 1)
grid(col = "grey85", lty = 1)

# improved (green)
plot(perf_i, add = TRUE,
     col = "#27AE60", lwd = 3, lty = 1)

abline(0, 1, col = "grey50", lty = 3, lwd = 1.5)

legend("bottomright",
      legend = c(paste0("Original   (AUC = ", round(auc_o, 3), ")"),
                  paste0("Improved   (AUC = ", round(auc_i, 3), ")"),
                  "Chance (0.500)"),
      col      = c("#E07A3F", "#27AE60", "grey50"),
      lwd      = c(3, 3, 1.5),
      lty      = c(2, 1, 3),
      bty      = "n",
      cex      = 0.9)
}
mtext("ROC Curves: Original vs Improved Bagging",
     outer = TRUE, cex = 1.3, font = 2)
par(mfrow = c(1, 1))

png("ROC_OriginalVsImproved.png",
    width = 1800, height = 600, res = 150)
par(mfrow = c(1, 3),
    mar = c(4.2, 4.2, 3.5, 1),
    oma = c(0, 0, 3, 0),
    mgp = c(2.3, 0.7, 0))
for (tgt in targets) {
  pred_o <- ROCR::prediction(orig_probs[[tgt]], true_labels[[tgt]])
  perf_o <- ROCR::performance(pred_o, "tpr", "fpr")
  auc_o <- as.numeric(pROC::auc(pROC::roc(true_labels[[tgt]],
                                          orig_probs[[tgt]],
                                          quiet = TRUE)))

  pred_i <- ROCR::prediction(improved_probs[[tgt]], true_labels[[tgt]])
  perf_i <- ROCR::performance(pred_i, "tpr", "fpr")
  auc_i <- as.numeric(pROC::auc(pROC::roc(true_labels[[tgt]],
                                          improved_probs[[tgt]],
                                          quiet = TRUE)))

  plot(perf_o, col = "#E07A3F", lwd = 3, lty = 2,
       main = paste("ROC -", tgt),
       xlab = "False Positive Rate", ylab = "True Positive Rate",
       las = 1)
  grid(col = "grey85", lty = 1)
  plot(perf_i, add = TRUE, col = "#2F6F9F", lwd = 3, lty = 1)
  abline(0, 1, col = "grey50", lty = 3, lwd = 1.5)
  legend("bottomright",
        legend = c(paste0("Original   (AUC = ", round(auc_o, 3), ")"),
                    paste0("Improved   (AUC = ", round(auc_i, 3), ")"),
                    "Chance (0.500)"),
        col      = c("#E07A3F", "#2F6F9F", "grey50"),
        lwd      = c(3, 3, 1.5), lty = c(2, 1, 3),
        bty      = "n", cex = 0.9)
}
mtext("ROC Curves: Original vs Improved Bagging",
     outer = TRUE, cex = 1.3, font = 2)
dev.off()
par(mfrow = c(1, 1))

```

```

# Per-target Original-vs-Improved summary
for (tgt in targets) {
  row_idx <- which(f1_compare$Target == tgt)
  cat("\n===== ", tgt, " =====\n", sep = "")
  cat("F1 Original:", f1_compare$F1_Original[row_idx],
      " | F1 Improved:", f1_compare$F1_Improved[row_idx],
      " | Delta:", f1_compare$Delta_F1[row_idx], "\n")
}

####Question 11 & quesiton 12 preparation - ANN
#find most country with most obeservations using clean and train set
sort(table(WD_Clean$Country), decreasing = TRUE)[1:10]
sort(table(WD.train$Country), decreasing = TRUE)[1:10]

#Step 1- select the country with most obeservations using train set
country_counts <- sort(table(WD.train$Country), decreasing = TRUE)
selected_country <- names(country_counts)[1]

cat("Country selected:", selected_country,
    "(", country_counts[1], "training rows)\n")

# check usa data distribution over different waves
table(WD.train[WD.train$Country == selected_country, ]$Wave)
table(WD.test [WD.test$Country == selected_country, ]$Wave)

ctry_train <- WD.train[WD.train$Country == selected_country, ]
ctry_test  <- WD.test[WD.test$Country == selected_country, ]

cat("Train rows:", nrow(ctry_train), "\n")
cat("Test rows :", nrow(ctry_test),  "\n")

#step 2: target + features
exists("top_features")
length(top_features)

target_var <- "CChurches"
ann_features <- top_features[1:10]
cat("Features:\n"); print(ann_features)

# ===== Step 3: Pre-processing =====
#get the data i would use from usa dataset (11)
ann_data <- ctry_train[, c(ann_features, target_var)]

# factor to numeric - attributes only
for (col in colnames(ann_data)) {
  if (is.factor(ann_data[[col]])) {
    ann_data[[col]] <- as.numeric(as.character(ann_data[[col]]))
  }
}

str(ann_data)
summary(ann_data)
# Normalize [0, 1] apply to only attributes not target
normalise <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}

```

```

}

norm_params <- lapply(ann_features, function(c) {
  v <- ann_data[[c]]
  list(min = min(v, na.rm = TRUE), max = max(v, na.rm = TRUE))
})
names(norm_params) <- ann_features

for (col in ann_features) {
  ann_data[[col]] <- normalise(ann_data[[col]])
}

# Indicator columns - true/false
ann_data$High <- ann_data$CChurches == 1
ann_data$Low <- ann_data$CChurches == 0

ann_data
# train ann model
ann_formula <- as.formula(
  paste("High + Low ~", paste(ann_features, collapse = " + "))
)

library(NeuralNetTools)
library(neuralnet)

#plan1: hidden = 3,
set.seed(33235422)
ctry_nn <- neuralnet(
  ann_formula, data = ann_data,
  hidden = 3, threshold = 0.05, stepmax = 1e6
)

# inspect weights
ctry_nn$result.matrix
print(ctry_nn$weights)
neuralweights(ctry_nn)

# plan2: hidden = c(8, 4)
set.seed(33235422)
ctry_nn_deep <- neuralnet(
  ann_formula, data = ann_data,
  hidden = c(8, 4), threshold = 0.05, stepmax = 1e6
)

plot(ctry_nn,
  rep = "best",
  information = TRUE,
  show.weights = TRUE,
  fontsize = 10,
  col.entry.synapse = "grey40",
  col.hidden.synapse = "grey40",
  col.out.synapse = "grey40",
  col.intercept = "#2F6F9F")

ctry_nn$result.matrix

plot(ctry_nn_deep, rep = "best")
ctry_nn_deep$result.matrix

```

```

####Q12 ANN performance across two waves (USA)
library(ROCR)
library(neuralnet)

# find out the 2 waves with most observations
wave_counts <- sort(table(ctry_test$Wave), decreasing = TRUE)
print(wave_counts)

top2_waves <- names(wave_counts)[1:2]
cat("Top 2 waves in test data:", top2_waves, "\n\n")

# container for waves
wave_metrics <- list()
wave_perf <- list()
wave_auc <- c()

# process 2 waves in for loop
for (w in top2_waves) {

  w_data <- ctry_test[ctry_test$Wave == w, ]
  cat("\n===== Wave", w, "=====\n")
  cat("N:", nrow(w_data), "\n")

  # conver to numeric matrix
  X <- w_data[, ann_features, drop = FALSE]
  for (col in colnames(X)) {
    if (is.factor(X[[col]]) || is.character(X[[col]])) {
      X[[col]] <- as.numeric(as.character(X[[col]]))
    }
  }
  # Apply training-time normalisation (use the SAME min/max as training)
  for (col in ann_features) {
    p <- norm_params[[col]]
    X[[col]] <- (X[[col]] - p$min) / (p$max - p$min)
  }
  X <- as.matrix(X)

  pred_full <- neuralnet::compute(ctry_nn, X)

  # find high column
  out_cols <- colnames(ctry_nn$response) # ("High", "Low")
  high_col_idx <- which(out_cols == "High")
  prob_high <- pred_full$net.result[, high_col_idx]
  pred_class <- ifelse(prob_high > 0.5, "High", "Low")

  # Match the training-time encoding High = CChurches == 1
  actual_num <- as.numeric(as.character(w_data$CChurches))
  actual <- ifelse(actual_num == 1, "High", "Low")

  # Confusion matrix
  cm <- table(Predicted = pred_class, Actual = actual)
  cat("\nConfusion Matrix:\n"); print(cm)

  cat(sprintf("Prob_High range: %.3f - %.3f (mean = %.3f)\n",
              min(prob_high), max(prob_high), mean(prob_high)))

  # F1 (positive class = "High")
  TP <- sum(pred_class == "High" & actual == "High")

```



```

FP <- sum(pred_class == "High" & actual == "Low")
FN <- sum(pred_class == "Low" & actual == "High")

precision <- ifelse((TP + FP) == 0, NA, TP / (TP + FP))
recall<- ifelse((TP + FN) == 0, NA, TP / (TP + FN))
f1 <- ifelse(is.na(precision) | is.na(recall) | (precision + recall) == 0,
            NA, 2 * precision * recall / (precision + recall))
accuracy <- mean(pred_class == actual)

# ROC + AUC
pred_obj <- ROCR::prediction(prob_high, actual_num)
perf <- ROCR::performance(pred_obj, "tpr", "fpr")
auc_val <- as.numeric(ROCR::performance(pred_obj, "auc")@y.values)

cat(sprintf("Acc=%.3f | Prec=%.3f | Rec=%.3f | F1=%.3f | AUC=%.3f\n",
            accuracy, precision, recall, f1, auc_val))

wave_metrics[[w]] <- data.frame(
  Wave = w, N = nrow(w_data),
  Accuracy = accuracy, Precision = precision,
  Recall = recall, F1 = f1, AUC = auc_val
)
wave_perf[[w]] <- perf
wave_auc[w] <- auc_val
}
summary_table

# summary table
summary_table <- do.call(rbind, wave_metrics)
rownames(summary_table) <- NULL
summary_table[, 3:7] <- round(summary_table[, 3:7], 3)
cat("\n===== Summary =====\n")
print(summary_table)

summary_table %>%
  kable(digits = 3,
        col.names = c("Wave", "N", "Accuracy", "Precision", "Recall", "F1",
"      AUC"),
        align = c("c", "c", rep("r", 5)),
        caption = "ANN performance across two waves (USA)") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
                full_width = T,
                position = "center") %>%
  row_spec(0, bold = TRUE, background = "#2F6F9F", color = "white")

#roc comaparison
png("Q12_ROC_wave_comparison.png", width = 1800, height = 1400, res = 200)
par(mar = c(5, 5, 4, 2))

plot(wave_perf[[top2_waves[1]]], col = "#E07A3F", lwd = 3, lty = 2,
     main = "ROC Curves: ANN performance across two waves (USA)",
     xlab = "False Positive Rate", ylab = "True Positive Rate",
     cex.main = 1.3, cex.lab = 1.1)
grid(col = "grey85", lty = 1)
plot(wave_perf[[top2_waves[2]]], add = TRUE, col = "#2F6F9F", lwd = 3, lty = 1)
abline(0, 1, col = "grey50", lty = 3, lwd = 1.5)

legend("bottomright",
      legend = c(
        paste0("Wave ", top2_waves[1], " (AUC = ",
round(wave_auc[top2_waves[1]], 3), ")"),

```

```

      paste0("Wave ", top2_waves[2], " (AUC = ",
round(wave_auc[top2_waves[2]], 3), ")"),
      "Random chance (0.500)"),
      col      = c("#E07A3F", "#2F6F9F", "grey50"),
      lwd      = c(3, 3, 1.5),
      lty      = c(2, 1, 3),
      cex      = 1.0,
      bg       = "white",
      box.col  = "grey70")
dev.off()

cat("\nROC plot saved to: Q12_ROC_wave_comparison.png\n")

head(ctry_nn$response)

X_train_mat <- as.matrix(ann_data[, ann_features])
pred_train  <- neuralnet::compute(ctry_nn, X_train_mat)$net.result
head(pred_train, 5)

# Compare with the training labels
head(ann_data[, c("High", "Low")], 5)

```